

COMPLETE EDITION · 2026

Android Keyboard Design Guide: *3D & Personalization*

A comprehensive companion to the Android Keyboard Design Guide. Covers every technique for building deeply customized keyboards — from OpenGL ES 3D rendering and Filament PBR shading to per-user dynamic themes, game engine bridges, AI-driven personalization, and production performance budgets. Verified API 30–36, full Kotlin/GLSL code examples.

2026 EDITION

3D & PERSONALIZATION

31 SECTIONS · FULL CODE

EXPERT 3D & PBR

FILAMENT · OPENGL ES · GLSL

MADE IN JURGISTAN

Companion to: Android Keyboard Design Guide

● **BEGINNER** Canvas drawing, XML layouts, basic theming

● **INTERMEDIATE** OpenGL ES setup, Compose shaders, custom rendering

● **EXPERT** Filament PBR, game engine bridges, real-time 3D in IME

SECTION 01

About This Volume

● **BEGINNER**

Scope, prerequisites, and how this extension fits into the broader keyboard design series

This guide extends the Android Keyboard Design Guide with a full deep-dive into **3D rendering, advanced personalization, and custom visual systems**.

Where the base guide establishes IME architecture, UX principles, and accessibility, this volume assumes you have that foundation and focuses exclusively on everything that makes a keyboard look, feel, and behave in ways that are uniquely yours.

A standard Android keyboard uses a `View` hierarchy rendered by the system's 2D canvas pipeline. Everything in this guide goes further: GPU-accelerated 3D geometry, physically-based rendering materials on individual keys, live particle effects on keystroke, user-up-loadable themes backed by a typed schema, and bridges to Unity and Godot for teams with existing game-engine expertise.

Prerequisites

- Working knowledge of `InputMethodService`, `onCreateInputView()`, and the IME lifecycle (see Section 05 of the base guide).
- Familiarity with Kotlin coroutines and Jetpack Compose basics.
- Basic understanding of how Android draws views (invalidate/draw cycle, hardware acceleration).

- For 3D sections (04–12): some exposure to coordinate systems and matrix math is helpful but not required — all math is explained inline.

Reading Paths



Designer / Theme Author

Start with [Section 17 Layout XML](#) → [Section 19 Theme Engine](#) → [Section 20 Backgrounds](#) → [Section 22 Fonts](#)



Android Engineer

Start with [Section 04 OpenGL ES](#) → [Section 05 GLSurfaceView](#) → [Section 13 Canvas](#) → [Section 28 Performance](#)



3D / Graphics Engineer

Start with [Section 07 GLSL Shaders](#) → [Section 09 Filament](#) → [Section 12 IBL](#) → [Section 25 AR Keyboard](#)



Game Developer

Start with [Section 23 Unity](#) → [Section 24 Godot](#) → [Section 14 Compose Shaders](#)



Performance Warning Up Front

3D rendering in an IME consumes GPU resources that compete with the foreground application. Every technique in this guide includes a performance budget note. Always profile on low-end devices (Snapdragon 4xx, MediaTek Helio G85) before shipping. See [Section 28](#) for the full budget framework.

SECTION 02

Rendering Architecture Overview

● INTERMEDIATE

Comparing all available rendering pipelines — from Android Canvas to Filament PBR — and choosing the right one

Android provides multiple paths for rendering a keyboard's UI. Your choice determines visual quality ceiling, complexity, and performance impact. This section maps every available option against a set of practical criteria.

RENDERING PATH	API LEVEL	3D CAPABLE	PBR	COMPOSE COMPATIBLE	GPU COST	BEST FOR
Android Canvas (2D)	API 21+	✗	✗	✓ Native	Very Low	Standard keyboard theming, flat design
Hardware-Accelerated Canvas	API 26+	Pseudo 3D only	✗	✓	Low	Shadows, layered drawables, bevel illusions
OpenGL ES 2.0 / 3.x + GLSurfaceView	API 21+	✓	Custom GLSL	Via TextureView	Medium	Full 3D keys, custom shaders, particle effects
Vulkan	API 24+	✓	✓	Via SurfaceView	Medium-High	Very high-end; rarely justified for keyboards
Filament (Google PBR)	API 21+	✓	✓ Full PBR	Via SurfaceView	Medium	Physically accurate materials, IBL lighting
Compose AGSL Shaders	API 33+	Pseudo 3D	Partial	✓ Native	Low-Medium	Animated gradients, blur, noise, distortion

Decision Framework

1. **Are you targeting all Android 5+ devices?** Use Canvas for base layers; OpenGL ES 2.0 for 3D overlays. Avoid Vulkan and AGSL shaders for broad support.
2. **Do you need physically accurate materials** (metallic keys, glass, rubber)? Filament is your only practical option without writing a full PBR renderer from scratch.
3. **Is your team already in Jetpack Compose?** Use AGSL shaders via `ShaderBrush` for effects; reserve OpenGL for genuine geometry.
4. **Are you shipping a game-engine-powered keyboard** (Unity / Godot)? See Sections [23](#) and [24](#) — the engine manages rendering entirely.



Recommended Hybrid Architecture

Production keyboards typically use a

Canvas base layer

for key outlines and labels (low cost, accessible, easy theming), with a

GLSurfaceView or TextureView overlay

for 3D effects, particle systems, or PBR highlights. The IME draws both surfaces through Android's compositor. This gives you the accessibility of Canvas rendering (TalkBack works, dynamic text scaling works) while layering real GPU effects on top.

SECTION 03

Open Source Reference IMEs

● BEGINNER

The essential open-source keyboard projects — what each teaches, and where to find advanced rendering examples

Before writing a single shader, spend time in these codebases. Each has solved architectural problems that would take months to discover independently. They are your best reference for how real keyboards handle theming, layout, and rendering at scale.

PROJECT	LICENSE	LANGUAGE	3D / VISUAL FEATURES	BEST LEARNING AREA
FlorisBoard (https://github.com/florisboard/florisboard)	Apache 2.0	Kotlin + Compose	Snygg theme engine, dynamic colors	Modern theme architecture, Compose IME integration
AnySoftKeyboard (https://github.com/AnySoftKeyboard/AnySoftKeyboard)	Apache 2.0	Java + Kotlin	3D theme pack (separate APK), NinePatch key drawables	Multi-language layout, subtype management, theme extension API
OpenBoard (https://github.com/openboard-team/openboard)	GPL 3.0	Java + Kotlin	Improved LatinIME rendering pipeline	LatinIME fork — XML layout system, hit-box math
kBoard (LiteKite/Android-IME) (https://github.com/LiteKite/Android-IME)	Apache 2.0	Kotlin	Clean architecture, MVVM	MVVM IME architecture, ViewModel in IME context
DonBrody/Android-CustomKeyboard (https://github.com/DonBrody/Android-CustomKeyboard)	MIT	Java	In-app keyboard (not IME)	Simplest possible custom keyboard — good starting point
shutterscripter/Custom_Android_Keyboard (https://github.com/shutterscripter/Custom_Android_Keyboard)	MIT	Java	Custom layout XML, themed keys	Layout XML customization patterns
AOSP LatinIME (https://android.googlesource.com/platform/packages/inputmethods/LatinIME)	Apache 2.0	Java/C++	KeyboardView canvas rendering	Gold standard reference — all layout and drawing math

AnySoftKeyboard's 3D Theme Pack — How It Works

AnySoftKeyboard 3D Theme (https://f-droid.org/en/packages/com.anysoftkeyboard.theme.three_d/) achieves its "3D buttons" look using a combination of **NinePatch drawables** with precisely crafted shadows and highlights baked into the PNG assets, and a custom `KeyboardView` subclass that adjusts the rendering order to paint shadow layers before key labels. This is a CPU-only approach — no GPU shaders — but it demonstrates that a convincing 3D appearance is achievable without OpenGL.

```
// From AnySoftKeyboard's theme extension pattern:
// 1. Each theme is a separate APK with its own drawable resources.
// 2. The host keyboard loads theme resources via PackageManager.
// 3. Key drawables follow the naming convention: key_normal.9.png,
//    key_pressed.9.png

// Loading a theme drawable from an external theme APK:
val themeContext = createPackageContext(
    themePackageName,
    Context.CONTEXT_IGNORE_SECURITY
)
val themeResources = themeContext.resources
val drawableId = themeResources.getIdentifier(
    "key_normal", "drawable", themePackageName
)
val keyDrawable = themeResources.getDrawable(drawableId, null)
```



Start with FlorisBoard's Snygg Engine

FlorisBoard's theme engine (called "Snygg") is the most architecturally modern open-source keyboard theme system available. It uses a typed JSON schema for themes, supports Material You dynamic colors, and has a functioning theme editor. Study the Snygg source directory (<https://github.com/florisboard/florisboard/tree/main/app/src/main/kotlin/dev/patrickgold/florisboard/snygg>) before building your own theme system.

OpenGL ES for Keyboards

● INTERMEDIATE

The OpenGL ES rendering model, coordinate systems, and why keyboards present unique integration challenges

OpenGL ES (OpenGL for Embedded Systems) is the primary low-level GPU API on Android. It is available on every Android device from API 21 onwards. Unlike desktop OpenGL, ES targets mobile GPUs — it has a reduced instruction set, stricter precision rules, and a programming model designed for tile-based deferred rendering architectures used by Qualcomm Adreno, ARM Mali, and Imagination PowerVR chips.

OpenGL ES Version Matrix

VERSION	ANDROID API	KEY FEATURES ADDED	KEYBOARD USE CASE
OpenGL ES 2.0	API 8+	Programmable shaders (vertex + fragment)	Basic 3D key geometry, simple materials
OpenGL ES 3.0	API 18+	Multiple render targets, instanced drawing, transform feedback	Particle systems, multi-pass rendering, instanced key drawing
OpenGL ES 3.1	API 21+	Compute shaders, SSBO, indirect draw	GPU-driven particle simulation, layout computation
OpenGL ES 3.2	API 24+	Geometry shaders, tessellation, advanced blend	Smooth curved key edges, advanced material blending

The Coordinate System

OpenGL ES uses a **right-handed coordinate system** with the origin at the center of the viewport. X increases to the right, Y increases upward, and Z increases toward the viewer. This differs from Android's View coordinate system where Y increases downward. You must account for this flip in every vertex shader that positions keyboard elements.

```
// Vertex shader – converting Android screen coords to OpenGL ES NDC
// Android: (0,0) top-left, Y down. OpenGL NDC: (-1,-1) bottom-left, Y up
attribute vec4 aPosition; // x, y in Android screen space (pixels)
uniform vec2 uResolution; // screen width, height
```

GLSL

```
uniform mat4 uMVP;           // model-view-projection matrix

void main() {
    // Normalize to [0, 1]
    vec2 normalised = aPosition.xy / uResolution;
    // Flip Y and remap to [-1, 1]
    vec2 clipSpace = (normalised * 2.0 - 1.0) * vec2(1.0, -1.0);
    gl_Position = uMVP * vec4(clipSpace, aPosition.zw);
}
```

Manifest Declaration

Declare the required OpenGL ES version in your `AndroidManifest.xml`. The IME `service` element does not need any changes — the declaration applies to the whole application.

```
<!-- In AndroidManifest.xml, inside <manifest> -->
<uses-feature
    android:glEsVersion="0x00030000"
    android:required="true" />
<!-- 0x00020000 = ES 2.0, 0x00030000 = ES 3.0, 0x00030002 = ES 3.2 -->
```



Setting required="true" limits your audience

Declaring a feature as required removes your app from the Play Store for devices that don't support it. For keyboards targeting the widest audience, set `android:required="false"` and check at runtime with `GLES30.glGetString(GLES30.GL_VERSION)`, falling back gracefully to Canvas rendering on unsupported devices.

GLSurfaceView in an IME

● INTERMEDIATE

How to embed a `GLSurfaceView` inside `InputMethodService`, lifecycle synchronization, and EGL context management

Embedding a `GLSurfaceView` inside an IME requires careful lifecycle management. Unlike an Activity, an IME has no `onPause()` / `onResume()` equivalents that automatically pause the GL thread. You must drive the OpenGL renderer lifecycle manually from the IME callbacks.

Full Integration Skeleton

```
// Full GLSurfaceView IME integration - Kotlin
class Keyboard3DService : InputMethodService() {

    private var glView: KeyboardGLSurfaceView? = null
    private var renderer: KeyboardRenderer? = null

    override fun onCreateInputView(): View {
        val renderer = KeyboardRenderer()
        this.renderer = renderer

        val glView = KeyboardGLSurfaceView(this).apply {
            setEGLContextClientVersion(3) // Request OpenGL ES 3.0
            setRenderer(renderer)
            renderMode = GLSurfaceView.RENDERMODE_WHEN_DIRTY
            // RENDERMODE_WHEN_DIRTY = render only when requestRender() is
            // Use RENDERMODE_CONTINUOUSLY only for animated effects
        }
        this.glView = glView
        return glView
    }

    override fun onStartInputView(info: EditorInfo, restarting: Boolean) {
        super.onStartInputView(info, restarting)
        glView?.onResume() // Resume the GL thread
    }
}
```

```

    override fun onFinishInputView(finishingInput: Boolean) {
        glView?.onPause() // Pause the GL thread when keyboard hides
        super.onFinishInputView(finishingInput)
    }

    override fun onDestroy() {
        renderer?.release() // Release OpenGL resources on service teardown
        super.onDestroy()
    }
}

// Custom GLSurfaceView that preserves the EGL context across hide/show cycles
class KeyboardGLSurfaceView(context: Context) : GLSurfaceView(context) {
    init {
        // IMPORTANT: preserve the EGL context so textures/shaders survive
        // keyboard hide/show cycles without full re-initialization
        setPreserveEGLContextOnPause(true)
    }
}

```

EGL Context Preservation

By default, a `GLSurfaceView` destroys its EGL context when paused. For an IME that is frequently hidden and shown (every time the user focuses a text field), this would cause full shader recompilation and texture re-upload on every appearance — adding 100–400 ms of lag. Calling `setPreserveEGLContextOnPause(true)` is **mandatory** for keyboard use.

Renderer Implementation Skeleton

```

class KeyboardRenderer : GLSurfaceView.Renderer {
    private var shaderProgram = 0
    private var keyVBO = 0
    private var viewWidth = 0
    private var viewHeight = 0

    override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
        // Called once (or after EGL context loss)
        // Safe to create shaders, upload textures, build VBOs here
        GLES30.glClearColor(0f, 0f, 0f, 0f) // Transparent background
        shaderProgram = buildShaderProgram(VERT_SRC, FRAG_SRC)
        keyVBO = buildKeyGeometry()
    }
}

```

KOTLIN

```

override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {
    // Called on creation and every rotation/resize
    viewWidth = width; viewHeight = height
    GLES30.glViewport(0, 0, width, height)
}

override fun onDrawFrame(gl: GL10?) {
    GLES30.glClear(GLES30.GL_COLOR_BUFFER_BIT or GLES30.GL_DEPTH_BUFFER_BIT)
    // Draw key geometry here
    drawKeys()
}

fun release() {
    // Called from service thread – must delete GL resources on GL thread
    glView?.queueEvent {
        GLES30.glDeleteProgram(shaderProgram)
        GLES30.glDeleteBuffers(1, intArrayOf(keyVB0), 0)
    }
}
}

```



GL resources must be created and destroyed on the GL thread

All OpenGL calls must happen on the GL thread, which is separate from the main/UI thread. Use `glView.queueEvent { }` to safely post work to the GL thread from any other thread. Never create or delete GL objects from the IME's main thread.

SECTION 06

3D Key Geometry & Meshes

● EXPERT

Building beveled, rounded, and extruded key meshes in code — vertex arrays, index buffers, and procedural geometry

A 3D keyboard key is fundamentally a **rectangular solid with rounded edges and a beveled top face**. This shape can be built procedurally from vertex data — no external 3D assets required. Understanding the geometry pipeline is prerequisite to all shader and lighting work.

Key Anatomy (3D)

Top Face Flat quad — label rendered here *Slightly inset from key edges*

Bevel Ring of quads around top face *~2–4dp wide, angled ~30°*

Side Walls 4 rectangular quads *Height ~4–8dp (key depth)*

Bottom Face Flat quad — usually hidden *Omit for performance*

Procedural Bevel Key Mesh

```
/**
 * Generates vertex data for a single 3D beveled key.
 *
 * Layout: [x, y, z, nx, ny, nz, u, v] per vertex (8 floats)
 * where (nx,ny,nz) is the surface normal and (u,v) is the UV coordinate.
 *
 * @param x      left edge in normalized device coordinates
 * @param y      top edge in NDC
 * @param w      width in NDC
 * @param h      height in NDC
 * @param depth  extrusion depth (z-axis) — try 0.04f for subtle 3D
 * @param bevel  bevel ring width — try 0.08f (fraction of min(w,h))
 */
```

KOTLIN

```

fun buildBevelKeyMesh(
    x: Float, y: Float, w: Float, h: Float,
    depth: Float = 0.04f,
    bevel: Float = 0.08f
): FloatArray {
    val bx = bevel * w    // bevel width on x axis
    val by = bevel * h    // bevel width on y axis

    // --- Top face (flat, faces +Z) ---
    // Outer bevel ring vertices (at full key footprint, z = depth * 0.5)
    // Inner top vertices (inset by bevel, z = depth)
    val outerZ = depth * 0.5f
    val topZ    = depth

    // 8 outer bevel corners + 4 inner top corners = 12 vertices for top
    // For brevity, using simplified rectangular approach (no corner rounding)
    return floatArrayOf(
        // Top face - 4 vertices (inner quad, facing +Z, normal = 0,0,1)
        x+bx,    -(y+by),    topZ,    0f,0f,1f,    0f,0f,
        x+w-bx, -(y+by),    topZ,    0f,0f,1f,    1f,0f,
        x+w-bx, -(y+h-by), topZ,    0f,0f,1f,    1f,1f,
        x+bx,    -(y+h-by), topZ,    0f,0f,1f,    0f,1f,

        // Front bevel face (bottom edge, facing -Y +Z at 45°)
        // normal = normalize(0, -1, 1)
        x+bx,    -(y+h-by), topZ,    0f,-0.707f,0.707f, 0f,1f,
        x+w-bx, -(y+h-by), topZ,    0f,-0.707f,0.707f, 1f,1f,
        x+w-bx, -(y+h),    outerZ,   0f,-0.707f,0.707f, 1f,0f,
        x+bx,    -(y+h),    outerZ,   0f,-0.707f,0.707f, 0f,0f
        // ... (repeat similar quads for left, right, top bevel faces)
        // ... (add 4 side wall quads from outerZ down to z=0)
    )
}

```

Instanced Drawing for All Keys

A keyboard has 30–50 keys. Drawing each with a separate draw call is expensive. OpenGL ES 3.0's **instanced drawing** lets you draw all keys with a single API call, passing per-key data (position, color, pressed state) as instance attributes.

```

// Instanced drawing – send all key transforms as a per-instance VBO
// instanceData: [x, y, width, height, r, g, b, isPressed] per key

val instanceCount = keyLayout.size

```

KOTLIN

```

val instanceData = FloatBuffer.allocate(instanceCount * 8)

keyLayout.forEachIndexed { i, key →
    instanceData.put(floatArrayOf(
        key.x, key.y, key.width, key.height,
        key.color.red, key.color.green, key.color.blue,
        if (key.isPressed) 1f else 0f
    ))
}
instanceData.rewind()

// Upload instance data to GPU
GLLES30.glBindBuffer(GLLES30.GL_ARRAY_BUFFER, instanceVB0)
GLLES30.glBufferData(GLLES30.GL_ARRAY_BUFFER,
    instanceData.capacity() * 4, instanceData, GLLES30.GL_DYNAMIC_DRAW)

// Draw all keys in one call
GLLES30.glDrawArraysInstanced(
    GLLES30.GL_TRIANGLES, 0, vertexCountPerKey, instanceCount
)

```

GLSL Shaders — Depth & Bevel

● EXPERT

Complete GLSL ES shader pairs for bevel lighting, press animation, and material surface effects

The shader is where a 3D keyboard comes alive. These are production-ready GLSL ES 3.0 shaders that implement a convincing beveled key appearance using analytical lighting — no texture baking required.

Vertex Shader — Key Surface with Press Animation

```
// vertex_key.glsl (GLSL ES 3.0 — #version 300 es)
#version 300 es

in vec3 aPosition;    // Key vertex position (model space)
in vec3 aNormal;      // Surface normal
in vec2 aUV;          // Texture coordinates

// Per-instance attributes (set divisor = 1)
in vec4 aKeyRect;     // x, y, width, height (NDC)
in vec3 aKeyColor;    // R, G, B
in float aIsPressed;  // 0.0 = normal, 1.0 = pressed

uniform mat4 uViewProjection;
uniform float uTime;    // Seconds since start (for animations)

out vec3 vWorldNormal;
out vec3 vWorldPos;
out vec2 vUV;
out vec3 vColor;
out float vPressAmt;

void main() {
    // Scale the unit key mesh to the actual key's rect dimensions
    vec3 scaledPos = aPosition * vec3(aKeyRect.z, aKeyRect.w, 1.0);

    // Press animation: sink key 60% of depth when pressed
    float pressDepth = aIsPressed * -0.6;
    scaledPos.z += pressDepth;
}
```

```

// Translate to key's screen position
vec3 worldPos = scaledPos + vec3(aKeyRect.xy, 0.0);

vWorldNormal = aNormal;
vWorldPos    = worldPos;
vUV          = aUV;
vColor       = aKeyColor;
vPressAmt    = aIsPressed;

gl_Position = uViewProjection * vec4(worldPos, 1.0);
}

```

Fragment Shader — Bevel Lighting + Material

```

// fragment_key.glsl (GLSL ES 3.0)
#version 300 es
precision mediump float;

in vec3  vWorldNormal;
in vec3  vWorldPos;
in vec2  vUV;
in vec3  vColor;
in float vPressAmt;

uniform vec3 uLightDir; // Normalized, world space
uniform vec3 uLightColor;
uniform float uRoughness; // 0=mirror 1=matte
uniform float uMetallic; // 0=plastic 1=metal
uniform sampler2D uNormalMap; // Optional normal map

out vec4 fragColor;

// Blinn-Phong lighting with specular highlight
vec3 blinnPhong(vec3 N, vec3 L, vec3 V, vec3 diffColor) {
    float NdotL = max(dot(N, L), 0.0);
    vec3 H = normalize(L + V);
    float specPow = mix(8.0, 256.0, 1.0 - uRoughness);
    float specular = pow(max(dot(N, H), 0.0), specPow);

    vec3 specColor = mix(vec3(1.0), diffColor, uMetallic);
    return diffColor * NdotL * uLightColor
        + specColor * specular * uLightColor * (1.0 - uRoughness);
}

```

```

void main() {
    vec3 N = normalize(vWorldNormal);
    vec3 L = normalize(uLightDir);
    vec3 V = vec3(0.0, 0.0, 1.0); // Camera direction (orthographic)

    // Ambient occlusion approximation for bevel faces (darker when facing away)
    float ao = mix(0.6, 1.0, dot(N, vec3(0.0,0.0,1.0)) * 0.5 + 0.5);

    vec3 lit = blinnPhong(N, L, V, vColor);

    // Darken on press – subtle shadow to reinforce physical metaphor
    float pressDim = 1.0 - vPressAmt * 0.18;

    fragColor = vec4(lit * ao * pressDim, 1.0);
}

```



Use mediump precision in fragment shaders

On mobile GPUs, `mediump float` (16-bit) is significantly faster than `highp` for fragment shaders. Keyboard lighting only needs ~3 decimal places of color precision. Reserve `highp` for vertex shaders where position math requires full 32-bit accuracy to prevent z-fighting.

Lighting Models for Keys

• EXPERT

Choosing and implementing the right lighting model — ambient, directional, IBL, and shadow approximation

Lighting transforms flat polygons into convincing surfaces. For a keyboard, the goal is not photorealism but **perceptual depth**: the viewer should intuitively understand which surface is the top face, which is the bevel, and which is the side wall — without needing to think about it.

Lighting Model Comparison

MODEL	GPU COST	REALISM	BEST FOR
Ambient only	Negligible	Flat / 2D	Not useful alone — but always add as base term
Lambertian diffuse	Very Low	Matte surface depth	Rubber / plastic keys, dark themes
Blinn-Phong	Low	Good specular highlights	Glossy plastic, light themes, most keyboards
Physically Based (PBR)	Medium	Metal, glass, rubber accurate	Premium keyboards; use Filament (Section 09)
IBL (Image-Based Lighting)	Medium-High	Environment reflection	Metallic / glass keys with Filament

Recommended Lighting Setup

For most keyboards, a **three-light setup** works well: one primary directional light from upper-left (simulating ambient room light), one soft fill from upper-right (prevents pure black shadows), and a subtle rim light from behind to define key edges. All three can be hard-coded as uniform vectors — no dynamic shadow maps required.

```
// Upload lighting uniforms from Kotlin (called once per frame or on theme change) KOTLIN
fun setLighting(program: Int, theme: KeyboardTheme) {
    // Primary: upper-left directional (60% contribution)
    GLES30.glUniform3f(
```

```

        GLES30.glGetUniformLocation(program, "uLightDir"),
        -0.5f, 0.7f, 0.6f // normalize(vec3(-0.5, 0.7, 0.6))
    )
    GLES30.glUniform3f(
        GLES30.glGetUniformLocation(program, "uLightColor"),
        theme.lightR, theme.lightG, theme.lightB
    )
    GLES30.glUniform1f(
        GLES30.glGetUniformLocation(program, "uRoughness"),
        theme.keyRoughness // e.g. 0.4 for glossy, 0.85 for matte
    )
    GLES30.glUniform1f(
        GLES30.glGetUniformLocation(program, "uMetallic"),
        theme.keyMetallic // e.g. 0.0 for plastic, 1.0 for chrome
    )
}

```

Filament Integration ● EXPERT

Embedding Google's Filament PBR engine inside an IME, Gradle setup, SurfaceView bridge, and engine lifecycle

Filament (<https://github.com/google/filament>) is Google's open-source, real-time physically based rendering engine. It targets OpenGL ES 3.x class GPUs and is the same engine used internally in Android Studio's design preview and the AR Sceneform SDK. It provides a full PBR material system, IBL lighting, shadow mapping, and a high-level scene graph — all in a library that loads quickly on mobile devices.

Gradle Dependencies

```
// build.gradle.kts (Module) – Filament for Android
dependencies {
    val filamentVersion = "1.74.0" // Latest stable as of 2026 Q1

    // Core Filament runtime
    implementation("com.google.android.filament:filament-
android:$filamentVersion")

    // High-level utils (ModelViewer, IBL etc.)
    implementation("com.google.android.filament:filament-utils-
android:$filamentVersion")

    // glTF 2.0 asset loading (for .glb key models)
    implementation("com.google.android.filament:gltfio-
android:$filamentVersion")

    // Material compiler at runtime (optional – use if generating materials on
device)
```

GRADLE

```
// implementation("com.google.android.filament:filament-
android:$filamentVersion")
}
```

Filament Engine Lifecycle in IME

```
class FilamentKeyboardService : InputMethodService() {
    private lateinit var engine: Engine
    private lateinit var renderer: Renderer
    private lateinit var scene: Scene
    private lateinit var view: View
    private lateinit var camera: Camera
    private var swapChain: SwapChain? = null
    private var surfaceView: SurfaceView? = null

    override fun onCreate() {
        super.onCreate()
        // Initialize Filament engine once – lives for the service's lifetime
        engine = Engine.create()
        renderer = engine.createRenderer()
        scene = engine.createScene()
        view = engine.createView()
        camera = engine.createCamera(engine.entityManager.create())

        view.scene = scene
        view.camera = camera
        setupOrthographicCamera()
    }

    override fun onCreateInputView(): View {
        val sv = SurfaceView(this)
        surfaceView = sv

        // SurfaceView callback – create SwapChain when surface is ready
        sv.holder.addCallback(object : SurfaceHolder.Callback {
            override fun surfaceCreated(holder: SurfaceHolder) {
                swapChain = engine.createSwapChain(holder.surface)
                buildKeyScene()
            }
            override fun surfaceChanged(h: SurfaceHolder, fmt: Int, w: Int, ht:
Int) {
                view.viewport = Viewport(0, 0, w, ht)
                setupOrthographicCamera(w.toFloat(), ht.toFloat())
            }
        })
    }
}
```

```

    }
    override fun surfaceDestroyed(h: SurfaceHolder) {
        swapChain?.let { engine.destroySwapChain(it) }
        swapChain = null
    }
})
return sv
}

private fun setupOrthographicCamera(w: Float = 1080f, h: Float = 400f) {
    // Orthographic projection maps pixel coords directly to NDC
    camera.setProjection(
        Camera.Projection.ORTHO,
        0.0, w.toDouble(), h.toDouble(), 0.0, 0.0, 10.0
    )
}

override fun onDestroy() {
    swapChain?.let { engine.destroySwapChain(it) }
    engine.destroyRenderer(renderer)
    engine.destroyScene(scene)
    engine.destroyView(view)
    engine.destroyCameraComponent(camera.entity)
    engine.destroy()
    super.onDestroy()
}
}

```



Filament Engine vs. Renderer vs. View

In Filament's architecture, the

Engine

is the factory and resource manager (one per process), the

Renderer

handles draw calls and frame timing, the

View

is a camera + scene pair that defines what to render and how, and the

SwapChain

is the surface being rendered into. Create the engine once in `onCreate()` but recreate the SwapChain every time the SurfaceView's surface is created/destroyed.

PBR Materials on Keys ● EXPERT

Authoring Filament materials (.filamat) for metallic, rubber, glass, and custom key surfaces

Filament's **material system** uses precompiled `.filamat` binary files that define a surface's physical properties. Each key can have a different material — metal for action keys, matte rubber for letter keys, glass for the spacebar. Materials are authored in a JSON-like `.mat` language and compiled offline with Filament's `matc` tool.

Material Source Files (.mat)

```
// key_standard.mat – Standard glossy plastic key material
material {
    name : "KeyPlastic",
    shadingModel : lit,
    blending : opaque,
    parameters : [
        { type : float3, name : baseColor },
        { type : float, name : roughness },
        { type : float, name : metallic },
        { type : float, name : pressAmt }
    ]
}

fragment {
    void material(inout MaterialInputs material) {
        prepareMaterial(material);

        // Slightly darken color on press
        float3 color = materialParams.baseColor
            * (1.0 - materialParams.pressAmt * 0.15);

        material.baseColor = float4(color, 1.0);
        material.roughness = materialParams.roughness;
        material.metallic = materialParams.metallic;
    }
}
```

```

        material.reflectance = 0.5; // Standard dielectric
    }
}

```

```

// key_metal.mat - Chrome / metallic action key
material {
    name : "KeyMetal",
    shadingModel : lit,
    parameters : [
        { type : float3, name : baseColor },
        { type : float, name : roughness }
    ]
}

fragment {
    void material(inout MaterialInputs material) {
        prepareMaterial(material);
        material.baseColor = float4(materialParams.baseColor, 1.0);
        material.metallic = 1.0; // Full metal
        material.roughness = materialParams.roughness;
        material.reflectance = 0.9; // High reflectance for chrome
    }
}

```

FILAMENT MATERIAL

Compiling with matc

```

# Run at build time (add to your build.gradle.kts as an exec task)
# matc ships with Filament - download from github.com/google/filament/releases

matc -a opengl -p mobile -o src/main/assets/key_standard.filamat
key_standard.mat
matc -a opengl -p mobile -o src/main/assets/key_metal.filamat key_metal.mat

```

BASH

Loading and Assigning Materials in Kotlin

```

fun loadKeyMaterial(engine: Engine, assetName: String): MaterialInstance {
    val bytes = assets.open(assetName).readBytes()
    val material = Material.Builder()
        .payload(bytes, bytes.size)
        .build(engine)
    return material.createInstance()
}

```

KOTLIN

```

}

// For each key, set material parameters and assign to renderable
fun applyMaterialToKey(
    engine: Engine,
    renderable: Int,
    materialInstance: MaterialInstance,
    keyColor: Color,
    isPressed: Boolean
) {
    materialInstance.setParameter("baseColor",
        RgbType.SRGB, keyColor.red, keyColor.green, keyColor.blue)
    materialInstance.setParameter("roughness", 0.35f)
    materialInstance.setParameter("metallic", 0.0f)
    materialInstance.setParameter("pressAmt", if (isPressed) 1.0f else 0.0f)

    RenderableManager.Builder(1)
        .material(0, materialInstance)
        .build(engine, renderable)
}

```

SECTION 11

SceneView & Compose Bridge

● EXPERT

Using SceneView (Filament + ARCore + Compose) as a higher-level alternative to raw Filament API calls

SceneView (<https://github.com/SceneView/scenview-android>) is an open-source library that wraps Google Filament with a Jetpack Compose-native API, including a `rememberEngine()` composable, collision system, and glTF model loader. If your team prefers Compose over raw GL, SceneView dramatically reduces the boilerplate of Sections 09 and 10.

Gradle Dependency

```
dependencies {  
    // SceneView - Filament + Compose bridge  
    implementation("io.github.scenview:scenview:2.2.1")  
    // AR variant (ARCore): io.github.scenview:arscenview:2.2.1  
}
```

GRADLE

3D Key Scene in Compose (inside ComposeView IME)

```
@Composable  
fun FilamentKeyboard(  
    keyLayout: List<KeyDef>,  
    pressedKeys: Set<Int>  
) {  
    val engine = rememberEngine()  
    val modelLoader = rememberModelLoader(engine)  
    val materialLoader = rememberMaterialLoader(engine)  
    val environmentLoader = rememberEnvironmentLoader(engine)  
  
    // Load key glTF model once and reuse for all keys  
    val keyModel by remember { mutableStateOf<null<ModelInstance>()>() }  
  
    Scene(  
        modifier = Modifier.fillMaxWidth().height(220.dp),  
        engine = engine,  
        modelLoader = modelLoader,  
        materialLoader = materialLoader,  
        environmentLoader = environmentLoader,  
        keyModel = keyModel,  
        keyLayout = keyLayout,  
        pressedKeys = pressedKeys  
    )  
}
```

KOTLIN

```

        modelLoader = modelLoader,
        materialLoader = materialLoader,
        environmentLoader = environmentLoader,
        environment = rememberEnvironment(environmentLoader) {
            environmentLoader.createHDREnvironment(
                assetFileLocation = "environments/studio_small.hdr"
            )!!
        },
        mainLightNode = rememberMainLightNode(engine) {
            intensity = 80_000.0f
        },
        childNodes = rememberNodes {
            keyLayout.mapIndexed { i, key →
                ModelNode(
                    modelInstance = modelLoader.createModelInstance(
                        assetFileLocation = "models/key_unit.glb"
                    ),
                    scaleToUnits = 0.9f
                ).apply {
                    position = Position(
                        x = key.centerX.toFloat(),
                        y = 0.0f,
                        z = if (i in pressedKeys) -0.15f else 0.0f
                    )
                }
            }
        }
    }
}

```



ComposeView in IME requires manual Lifecycle wiring

A `ComposeView` inside `onCreateInputView()` must be wired to a `LifecycleOwner` and `SavedStateRegistryOwner` via `ViewTreeLifecycleOwner.set()` and `ViewTreeSavedStateRegistryOwner.set()`. Without this, composables that use `rememberCoroutineScope`, `LaunchedEffect`, or any lifecycle-aware state will crash or silently fail. See [this detailed Compose-in-IME tutorial](https://medium.com/@maksymkovali/implementation-of-a-custom-soft-keyboard-in-android-using-compose-b8522d7ed9cd) (<https://medium.com/@maksymkovali/implementation-of-a-custom-soft-keyboard-in-android-using-compose-b8522d7ed9cd>) for the complete wiring pattern.

SECTION 12

IBL Environment Lighting

● EXPERT

Image-based lighting for metallic and glass keys — HDR environment maps, KTX packaging, and tone mapping

Image-based lighting (IBL) uses a 360° HDR environment map to provide realistic ambient and specular reflections. On metallic or glass key materials, IBL transforms flat-looking geometry into convincing mirror-like surfaces that reflect the environment. With Filament, IBL setup requires a pre-filtered KTX file generated offline.

Generating KTX Environment Maps

```
# Step 1: Download Filament tools from github.com/google/filament/releases
# Step 2: Use cmgen to convert an HDR panorama to Filament's KTX format

# Download a free HDR from polyhaven.com (e.g., studio_small_09.hdr)
cmgen \
  --format=ktx \
  --size=256 \
  --deploy=./src/main/assets/environments/ \
  studio_small_09.hdr

# Output: ./assets/environments/studio_small_09/
#   studio_small_09_ibl.ktx (specular irradiance, pre-filtered mips)
#   studio_small_09_skybox.ktx (skybox cubemap)
```

BASH

Loading IBL in Kotlin

```
fun loadEnvironment(engine: Engine, assetPrefix: String): IndirectLight {
    val iblBuffer = assets.open("$assetPrefix_ibl.ktx").readBytes()
        .let { ByteBuffer.wrap(it) }

    val ibl = KtxLoader.createIndirectLight(engine, iblBuffer,
        KtxLoader.Options())
}
```

KOTLIN

```
ibl.intensity = 30_000f // Tuned for keyboard size
return ibl
}
```



Match IBL intensity to light theme / dark theme

On light themes, IBL intensity of 30,000–50,000 produces realistic metallic keys. On dark themes, reduce to 10,000–20,000 to prevent overexposure. Expose this as a theme token (`theme.iblIntensity`) so users with custom backgrounds can tune it.

Tone Mapping: HDR to Display Range

After IBL produces high-dynamic-range lighting values, a **tone mapping operator** compresses the luminance range into displayable [0, 1] values. Filament defaults to **ACES** (Academy Color Encoding System), the film-industry standard. For keyboard surfaces where metallic reflections dominate, ACES preserves specular highlight detail better than simpler operators like Reinhard.

```
// Configure tone mapping in Filament View settings
fun configureToneMapping(view: com.google.android.filament.View) {
    // ACES: Best for metallic / reflective key materials
    view.colorGrading = ColorGrading.Builder()
        .toneMapping(ColorGrading.ToneMapping.ACES)
        .exposure(0.0f) // Neutral – IBL intensity handles brightness
        .nightAdaptation(0.0f)
        .build(engine)

    // Alternative: FILMIC for a flatter, more predictable look (matte themes)
    // view.colorGrading = ColorGrading.Builder()
```

KOTLIN

```
//      .toneMapping(ColorGrading.ToneMapping.FILMIC)
//      .build(engine)
}
```

IBL Format Comparison

FORMAT	HDR SUPPORT	MIP LEVELS	ANDROID GPU	SIZE (256 ²)	TOOL
KTX2 + Basis	Yes (BC6H / ASTC HDR)	Pre-filtered	✅ GPU-native decode	~150 KB	cmgen (https://github.com/google/filament/releases)
Equirect HDR	Yes (.hdr / .exr)	None (raw panorama)	❌ CPU decode first	~2 MB	Poly Haven (https://polyhaven.com/hdris)
RGBM Cubemap	Encoded (8-bit + multiplier)	Manual	✅ Standard texture	~400 KB	Custom pipeline
Spherical Harmonics (L2)	Diffuse only	N/A (9 coefficients)	✅ Trivial (27 floats)	~108 bytes	cmgen --sh-compute (https://github.com/KhronosGroup/KTX-Software)

Spherical Harmonics for Fast Diffuse Lighting

For keys with primarily *matte* materials (non-metallic), full cubemap IBL is overkill.

Spherical Harmonics (SH) encode the low-frequency lighting environment in just 9 coefficients per color channel — 27 floats total. This provides smooth diffuse ambient lighting at near-zero GPU cost. Filament's `cmgen` outputs SH data alongside the cubemap:

```
# Generate spherical harmonics alongside the KTX cubemap
cmgen \
  --format=ktx \
  --size=256 \
  --sh-compute=3 \
  --deploy=./src/main/assets/environments/ \
  studio_small_09.hdr

# Output includes sh.txt: 9 RGB coefficients (L0, L1, L2 bands)
# Load as a FloatArray and pass to IndirectLight.Builder
```

BASH

```

// Load spherical harmonics from a pre-computed float array
fun loadDiffuseOnlyIBL(engine: Engine, shCoeffs: FloatArray): IndirectLight {
    // shCoeffs: 27 floats (9 coefficients × 3 RGB channels)
    require(shCoeffs.size == 27) { "Expected 27 SH coefficients" }

    return IndirectLight.Builder()
        .irradiance(3, shCoeffs) // 3 bands = L0 + L1 + L2
        .intensity(30_000f)
        .build(engine)
}

```

Runtime Environment Switching

A theme-aware keyboard should swap IBL environments when the user toggles between light and dark mode — or when ambient brightness changes. Pre-load both environments at initialization and switch at zero runtime cost:

```

class IBLManager(
    private val engine: Engine,
    private val scene: Scene,
    private val assets: AssetManager
) {
    private var lightEnv: IndirectLight? = null
    private var darkEnv: IndirectLight? = null

    fun preload() {
        lightEnv = loadEnvironment(engine, "environments/studio_bright")
            .apply { intensity = 40_000f }
        darkEnv = loadEnvironment(engine, "environments/night_soft")
            .apply { intensity = 12_000f }
    }

    fun applyForMode(isDark: Boolean) {
        scene.indirectLight = if (isDark) darkEnv else lightEnv
    }

    fun destroy() {
        lightEnv?.let { engine.destroyIndirectLight(it) }
        darkEnv?.let { engine.destroyIndirectLight(it) }
    }
}

```



GPU Memory Budget for KTX Cubemaps

A 256×256-face KTX cubemap with mipmaps consumes approximately

2–3 MB of GPU memory

. Pre-loading both light and dark environments doubles this. On entry-level GPUs (Mali-G52, Adreno 610), keep total IBL memory under 6 MB and use 128×128 faces — the visual difference on small keyboard surfaces is negligible. Always call `engine.destroyIndirectLight()` when the keyboard is hidden to free GPU memory.

Further Reading

- **Filament Documentation — Image-Based Lights** (<https://google.github.io/filament/Filament.html#imagingpipeline/lightpath/imagebasedlights>)
— Complete reference for IBL in Filament, including spherical harmonics and cubemap processing.
- **Poly Haven — Free HDR Environment Maps** (<https://polyhaven.com/hdris>) — CCO-licensed 360° HDR panoramas ready for `cmgen` conversion to KTX.
- **Khronos KTX-Software** (<https://github.com/KhronosGroup/KTX-Software>) — Official toolchain for creating, reading, and transcoding KTX v2 textures.
- **LearnOpenGL — IBL Diffuse Irradiance** (<https://learnopengl.com/PBR/IBL/Diffuse-irradiance>) — Walkthrough of spherical harmonics and irradiance convolution from first principles.

Canvas + Hardware Acceleration ● INTERMEDIATE

Achieving pseudo-3D depth, custom drawables, and hardware-layer compositing without any OpenGL code

Not every 3D effect requires writing shaders. Android's hardware-accelerated Canvas, combined with carefully crafted drawables, can produce convincing depth through **layered shadows, gradient highlights, and strategic compositing**. This approach is accessible to every Android developer and runs on all API 21+ devices with no special permissions.

Hardware-Accelerated Canvas Layers

```
class Pseudo3DKeyView(context: Context) : View(context) {
    private val shadowPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.rgb(60, 0, 0)
        maskFilter = BlurMaskFilter(8f, BlurMaskFilter.Blur.NORMAL)
    }
    private val keyPaint = Paint(Paint.ANTI_ALIAS_FLAG)
    private val highlightPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        shader = null // Assigned per-draw with LinearGradient
    }

    override fun onDraw(canvas: Canvas) {
        val rect = RectF(8f, 8f, width - 8f, height - 8f)

        // 1. Drop shadow (draw before key body)
        val shadowRect = RectF(rect.left + 2f, rect.top + 4f,
                               rect.right + 2f, rect.bottom + 4f)
        canvas.drawRoundRect(shadowRect, 8f, 8f, shadowPaint)

        // 2. Key body with flat color
        keyPaint.color = keyColor
        canvas.drawRoundRect(rect, 8f, 8f, keyPaint)

        // 3. Gradient highlight simulating curved top surface
        //    Top half: lighter (catching top light)
    }
}
```

```

// Bottom half: darker (shadow in lower bevel)
highlightPaint.shader = LinearGradient(
    rect.left, rect.top, rect.left, rect.bottom,
    intArrayOf(
        Color.argb(80, 255, 255, 255), // highlight
        Color.argb(0, 255, 255, 255), // fade out
        Color.argb(40, 0, 0, 0), // shadow
    ),
    floatArrayOf(0f, 0.5f, 1f),
    Shader.TileMode.CLAMP
)
canvas.drawRoundRect(rect, 8f, 8f, highlightPaint)

// 4. Key label via TextPaint
drawLabel(canvas, rect)
}
}

```

Hardware Layer for Animated Keys

For animated keys (ripple on press, glow on hover), use `setLayerType(LAYER_TYPE_HARDWARE, null)`. This caches the view's rendered output in a GPU texture, making subsequent redraws (alpha fade, translation) GPU-only operations with near-zero CPU cost.

```

// Enable hardware layer for a key that will be animated
keyView.setLayerType(View.LAYER_TYPE_HARDWARE, null)

// Press animation – GPU-only translation, no redraw needed
ObjectAnimator.ofFloat(keyView, "translationY", 0f, 2.5f).apply {
    duration = 60
    interpolator = AccelerateInterpolator()
    start()
}

```

KOTLIN

BlendMode Compositing for Glass and Frosted Keys

Android 29+ (API 29) introduced `BlendMode` via `android.graphics.BlendMode` (<https://developer.android.com/reference/android/graphics/BlendMode>), replacing the legacy `PorterDuff.Mode`. For keyboard keys, blending enables frosted-glass overlays, inner glows, and light-bleed effects — all composited by the GPU without touching OpenGL.

```
// Frosted-glass key overlay using BlendMode (API 29+)
private val frostPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
    color = Color.rgb(100, 255, 255, 255)
    blendMode = BlendMode.SOFT_LIGHT    // Gentle highlight layer
}

private val innerGlowPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
    color = Color.rgb(30, 255, 255, 255)
    maskFilter = BlurMaskFilter(12f, BlurMaskFilter.Blur.INNER)
    blendMode = BlendMode.SCREEN        // Additive inner glow
}

override fun onDraw(canvas: Canvas) {
    // 1. Draw key body (opaque)
    canvas.drawRoundRect(keyRect, 12f, 12f, keyPaint)
    // 2. Frost overlay – blends with underlying body
    canvas.drawRoundRect(keyRect, 12f, 12f, frostPaint)
    // 3. Inner glow – additive light bleed along edges
    canvas.drawRoundRect(keyRect, 12f, 12f, innerGlowPaint)
    // 4. Label last (always on top)
    drawLabel(canvas, keyRect)
}
```

KOTLIN

BlendMode Quick Reference

BLENDMODE	VISUAL EFFECT	KEYBOARD USE CASE
SOFT_LIGHT	Gentle highlight / darken	Frosted-glass key overlay
SCREEN	Additive (lighten only)	Inner glow, neon edges
MULTIPLY	Darken (shadow areas)	Key underside shadow
OVERLAY	Contrast-boosting blend	Embossed label illusion
COLOR_DODGE	Bright highlight punch	Specular hotspot on metallic keys

Path Effects for Embossed & Engraved Labels

Embossed key labels create a convincing 3D text illusion using two offset text draws — one for the highlight (top-left) and one for the shadow (bottom-right). This technique works on any API level and requires no shaders.

```

fun drawEmbossedLabel(canvas: Canvas, text: String, cx: Float, cy: Float) { KOTLIN
    val highlightPaint = TextPaint(Paint.ANTI_ALIAS_FLAG).apply {
        textSize = 28f
        color = Color.argb(120, 255, 255, 255) // Light top edge
        textAlign = Paint.Align.CENTER
    }
    val shadowPaint = TextPaint(highlightPaint).apply {
        color = Color.argb(80, 0, 0, 0) // Dark bottom edge
    }
    val mainPaint = TextPaint(highlightPaint).apply {
        color = keyLabelColor
    }

    // 1 px offset creates emboss illusion at standard keyboard densities
    canvas.drawText(text, cx - 1f, cy - 1f, highlightPaint) // highlight
    canvas.drawText(text, cx + 1f, cy + 1f, shadowPaint) // shadow
    canvas.drawText(text, cx, cy, mainPaint) // main label
}

```

Canvas vs OpenGL: When to Upgrade

CRITERION	CANVAS (HARDWARE-ACCELERATED)	OPENGL ES / FILAMENT
Depth layers	2–4 layers (shadow + body + highlight + label)	Unlimited (true z-buffer)
Specular reflections	Gradient approximation only	Physically-based (PBR)
Animated lighting	No (static gradient baked)	Real-time per-pixel shading
Key press depth	Translation + shadow resize	True mesh displacement
Min API	21 (full hardware accel)	24 (GLES 3.1) / 28 (Filament)
Battery impact	Minimal (GPU idle between draws)	Continuous render loop



Minimize `invalidate()` calls on Canvas keyboards

Each `invalidate()` triggers a full `onDraw()` pass for the entire keyboard view. During rapid typing, this can saturate the GPU command queue. Instead, use `invalidate(Rect)` to dirty only the pressed key's bounds, or switch to `LAYER_TYPE_HARDWARE` for animated keys so that press feedback uses cached GPU textures rather than redrawing the full keyboard.

Further Reading

- **Android Developers — Hardware Acceleration** (<https://developer.android.com/develop/ui/views/graphics/hardware-accel>)
— Official guide to Canvas hardware acceleration, supported operations, and layer types.
- **BlendMode API Reference** (<https://developer.android.com/reference/android/graphics/BlendMode>) —
Complete enum of all blend modes with visual descriptions.
- **Optimizing View Hierarchies** (<https://developer.android.com/topic/performance/rendering/optimizing-view-hierarchies>)
— Performance patterns for custom views, including layer caching and selective invalidation.

ComposeUI Shader Effects — AGSL ● INTERMEDIATE

Android Graphics Shading Language (AGSL) in Jetpack Compose: animated gradients, noise textures, blur, and distortion on key surfaces

Android 13 (API 33) introduced **AGSL** — Android Graphics Shading Language — a shader dialect similar to GLSL that runs on Android's 2D rendering pipeline. In Jetpack Compose, AGSL shaders are applied via `ShaderBrush`, enabling effects like animated noise, neon glow, refraction, and liquid metal key surfaces — all without leaving Compose.

Animated Noise Key Background

```
KOTLIN
val animatedNoiseShader = remember {
    RuntimeShader("""
        uniform float2  iResolution;
        uniform float    iTime;
        uniform float3   iBaseColor;

        // Classic 2D value noise
        float hash(float2 p) {
            return fract(sin(dot(p, float2(127.1, 311.7)))) * 43758.5453);
        }
        float noise(float2 p) {
            float2 i = floor(p);
            float2 f = fract(p);
            f = f*f*(3.0 - 2.0*f); // Smoothstep
            return mix(
                mix(hash(i),          hash(i + float2(1,0)), f.x),
                mix(hash(i+float2(0,1)), hash(i + float2(1,1)), f.x),
                f.y
            );
        }

        float4 main(float2 fragCoord) {
            float2 uv = fragCoord / iResolution;
            // Animate noise over time for a shimmering effect
            float n = noise(uv * 6.0 + float2(iTime * 0.4, 0.0));
```

```

        float3 col = iBaseColor + n * 0.06; // Subtle surface variation
        return float4(col, 1.0);
    }
    """.trimIndent()
}

val time by produceState(0f) {
    while (true) {
        withFrameNanos { t → value = t / 1_000_000_000f }
    }
}

animatedNoiseShader.setFloatUniform("iTime", time)
animatedNoiseShader.setFloatUniform("iBaseColor", 0.2f, 0.3f, 0.8f)

Box(modifier = Modifier
    .background(ShaderBrush(animatedNoiseShader))
)

```



AGSL requires API 33 — gate carefully

`RuntimeShader` and `ShaderBrush` are only available from Android 13 (API 33). Always check `Build.VERSION.SDK_INT ≥ Build.VERSION_CODES.TIRAMISU` before creating a `RuntimeShader`, and provide a plain gradient fallback for older devices. Never let an AGSL path reach devices running API 30–32.

AGSL vs GLSL ES: Key Syntax Differences

AGSL is based on GLSL ES 3.0 but has important differences. Knowing these upfront prevents frustrating shader compilation errors at runtime.

FEATURE	GLSL ES 3.0	AGSL
Entry point	<code>void main()</code>	<code>half4 main(float2 fragCoord)</code>
Output	<code>gl_FragColor</code> / out variable	Return value (<code>half4</code>)
Color space	Linear RGB assumed	Color-managed (sRGB by default)
Precision	<code>mediump</code> , <code>highp</code>	<code>half</code> (mediump), <code>float</code> (highp)
Texture sampling	<code>texture(sampler, uv)</code>	<code>child.eval(uv)</code> via shader children
Uniforms	Standard GLSL uniforms	<code>uniform float name;</code> — set via <code>setFloatUniform()</code>

Neon Glow Edge Shader

A popular effect for futuristic keyboard designs: keys with glowing neon borders. This AGSL shader computes signed distance from a rounded rectangle and applies an exponential glow falloff.

```

// Neon glow rounded-rect — set uniforms from Kotlin
uniform float2 iResolution;    // Key width, height in px
uniform float iCornerRadius;    // Rounded corner radius
uniform float3 iGlowColor;     // Neon RGB (e.g. 0.0, 1.0, 0.8)
uniform float iGlowWidth;     // Glow spread in px (8-16)
uniform float iTime;           // Animate pulse

float sdRoundRect(float2 p, float2 b, float r) {
    float2 d = abs(p) - b + r;
    return length(max(d, 0.0)) + min(max(d.x, d.y), 0.0) - r;
}

half4 main(float2 fragCoord) {
    float2 uv = fragCoord - iResolution * 0.5;
    float dist = sdRoundRect(uv, iResolution * 0.5, iCornerRadius);
    float pulse = 0.8 + 0.2 * sin(iTime * 3.0);
    float glow = pulse * exp(-dist * dist / (iGlowWidth * iGlowWidth));
    half3 col = half3(iGlowColor) * half(glow);

```

```

half alpha = half(smoothstep(1.0, 0.0, dist));
return half4(col, alpha);
}

```

Liquid Chrome Reflection

Chrome effects simulate metallic surfaces by distorting a reflected environment based on surface normals. The shader below creates a warped metallic reflection that responds to time — ideal for premium key themes.

```

uniform float iTime;
uniform float2 iResolution;
uniform float3 iChromeColor; // Base tint (0.8, 0.85, 0.9 for silver)

half4 main(float2 fragCoord) {
    float2 uv = fragCoord / iResolution;
    // Distort UVs to simulate curved surface
    float dx = sin(uv.y * 12.0 + iTime) * 0.02;
    float dy = cos(uv.x * 10.0 + iTime * 0.7) * 0.015;
    float2 distorted = uv + float2(dx, dy);

    // Fresnel-like edge brightening
    float edge = pow(1.0 - abs(uv.x - 0.5) * 2.0, 3.0);
    float streak = smoothstep(0.48, 0.52, distorted.y);

    half3 col = half3(iChromeColor) * half(0.6 + 0.4 * streak + 0.2 * edge);
    return half4(col, 1.0);
}

```

Versioned Fallback Pattern

Because AGSL requires API 33, every shader effect must have a graceful fallback. The recommended pattern wraps creation in a version check and uses a simple gradient or solid color on older devices.

```

object ShaderCompat {

    /**
     * Returns a RuntimeShader for API 33+ or null for older devices.
     * Caller must provide a Brush/Paint fallback when null is returned.
     */
    fun createIfSupported(agslSource: String): RuntimeShader? {

```

```

        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU) {
            return null
        }
        return try {
            RuntimeShader(agslSource)
        } catch (e: IllegalArgumentException) {
            // Shader compilation failed – log and fall back
            Log.w("ShaderCompat", "AGSL compile failed", e)
            null
        }
    }
}

// Usage in a Composable
val shader = remember { ShaderCompat.createIfSupported(NEON_GLOW_AGSL) }
val brush = if (shader != null) {
    ShaderBrush(shader)
} else {
    Brush.linearGradient(listOf(glowColor, Color.Transparent))
}

```

Performance Profiling AGSL Shaders



Profile each shader's GPU cost

Use

GPU Profiler

in Android Studio (via *View* → *Tool Windows* → *Profiler* → *GPU*) to identify expensive fragment operations. AGSL shaders compile to Skia pipelines that may run as software rasterization on some Mali GPUs. Key metrics to watch:

— Frame render time

- must stay under 16 ms (60 fps), ideally under 8 ms (120 fps).

— Shader compile latency

- `RuntimeShader()` constructor compiles synchronously; do it off the main thread or in `remember {}`.

— Uniform updates

- `setFloatUniform()` is cheap, but changing the shader source triggers recompilation.

Further Reading

- [Android Developers — AGSL Overview](https://developer.android.com/develop/ui/views/graphics/agsl) (<https://developer.android.com/develop/ui/views/graphics/agsl>) — Official introduction to the Android Graphics Shading Language.
 - [AGSL vs GLSL Differences](https://developer.android.com/develop/ui/views/graphics/agsl/agsl-vs-glsl) (<https://developer.android.com/develop/ui/views/graphics/agsl/agsl-vs-glsl>) — Side-by-side comparison of AGSL and GLSL ES syntax.
 - [Shadertoy](https://www.shadertoy.com/) (<https://www.shadertoy.com/>) — Community shader gallery; many GLSL effects can be ported to AGSL with the syntax changes from the table above.
 - [2D SDF Functions by Inigo Quilez](https://iquilezles.org/articles/distfunctions2d/) (<https://iquilezles.org/articles/distfunctions2d/>) — Comprehensive signed-distance-function reference used for rounded-rect, circle, and polygon key shapes in AGSL shaders.
-

TextureView Overlay Techniques ● INTERMEDIATE

Using `TextureView` to layer live video, camera feeds, or OpenGL output behind a Canvas key layer

A `TextureView` renders into an Android `SurfaceTexture`, which can be drawn anywhere in the View hierarchy — including *underneath* a Canvas key layer. This makes it possible to have a **live video background**, a camera passthrough, or an OpenGL render visible behind your keyboard keys.

Architecture: TextureView under Canvas Keys

```
KOTLIN
override fun onCreateInputView(): View {
    // FrameLayout: TextureView (background) + KeyboardCanvas (foreground)
    val root = FrameLayout(this)

    // 1. TextureView – live 3D/video background
    val textureView = TextureView(this).apply {
        setOpaque(false) // Transparent so keys are visible on top
        surfaceTextureListener = backgroundSurfaceListener
    }

    // 2. Canvas keyboard layer (transparent background)
    val keyCanvas = KeyboardCanvasView(this).apply {
        setBackgroundColor(Color.TRANSPARENT)
    }

    root.addView(textureView, FrameLayout.LayoutParams.MATCH_PARENT,
        FrameLayout.LayoutParams.MATCH_PARENT)
    root.addView(keyCanvas, FrameLayout.LayoutParams.MATCH_PARENT,
        FrameLayout.LayoutParams.MATCH_PARENT)

    return root
}

private val backgroundSurfaceListener = object :
    TextureView.SurfaceTextureListener {
    override fun onSurfaceTextureAvailable(st: SurfaceTexture, w: Int, h: Int) {
        // Start OpenGL render loop targeting this surface
        backgroundRenderer.start(Surface(st), w, h)
    }
}
```

```

    override fun onSurfaceTextureDestroyed(st: SurfaceTexture): Boolean {
        backgroundRenderer.stop(); return true
    }
    override fun onSurfaceTextureSizeChanged(st: SurfaceTexture, w: Int, h: Int)
    {}
    override fun onSurfaceTextureUpdated(st: SurfaceTexture) {}
}

```

Surface Type Comparison

Android offers three surface types for GPU-rendered content. Choosing the right one determines compositing flexibility, power draw, and input latency.

FEATURE	SURFACEVIEW	TEXTUREVIEW	GLSURFACEVIEW
Compositing	Separate window (below or above app)	Normal view in hierarchy	Separate window (like SurfaceView)
Transforms (alpha, rotation)	No — separate layer	Yes — full View transforms	No — separate layer
Transparent overlay	Complex (Z-order hacks)	Simple (<code>setOpaque(false)</code>)	Complex
EGL context management	Manual	Manual	Automatic (via <code>GLSurfaceView.Renderer</code>)
Performance overhead	Lowest (direct compositor)	~5–15% GPU extra (texture copy)	Low (direct compositor)
Best for keyboard use	Full-screen 3D keyboard	Background effect under Canvas keys	Dedicated 3D keyboard view

EGL Context Setup for the Render Thread

When using `TextureView` (or `SurfaceView`), you must create and manage your own EGL context on a dedicated render thread. The following skeleton shows the standard initialization sequence.

```

class BackgroundRenderer {

    private var eglDisplay: EGLDisplay = EGL14.EGL_NO_DISPLAY
    private var eglContext: EGLContext = EGL14.EGL_NO_CONTEXT
    private var eglSurface: EGLSurface = EGL14.EGL_NO_SURFACE
    private var running = false

    fun start(surface: Surface, width: Int, height: Int) {

```

KOTLIN

```

        running = true
        Thread {
            initEGL(surface)
            GLES30.glViewport(0, 0, width, height)
            while (running) {
                drawFrame()
                EGL14.eglSwapBuffers(eglDisplay, eglSurface)
            }
            releaseEGL()
        }.start()
    }

    private fun initEGL(surface: Surface) {
        eglDisplay = EGL14.eglGetDisplay(EGL14.EGL_DEFAULT_DISPLAY)
        val version = IntArray(2)
        EGL14.eglInitialize(eglDisplay, version, 0, version, 1)

        val attribs = intArrayOf(
            EGL14.EGL_RENDERABLE_TYPE, EGL14.EGL_OPENGL_ES3_BIT,
            EGL14.EGL_RED_SIZE, 8,
            EGL14.EGL_GREEN_SIZE, 8,
            EGL14.EGL_BLUE_SIZE, 8,
            EGL14.EGL_ALPHA_SIZE, 8, // Needed for transparency
            EGL14.EGL_DEPTH_SIZE, 16,
            EGL14.EGL_NONE
        )
        val configs = arrayOfNulls<EGLConfig>(1)
        val numConfigs = IntArray(1)
        EGL14.eglChooseConfig(eglDisplay, attribs, 0, configs, 0, 1, numConfigs,
0)

        val ctxAttribs = intArrayOf(EGL14.EGL_CONTEXT_CLIENT_VERSION, 3,
EGL14.EGL_NONE)
        eglContext = EGL14.eglCreateContext(
            eglDisplay, configs[0], EGL14.EGL_NO_CONTEXT, ctxAttribs, 0
        )
        eglSurface = EGL14.eglCreateWindowSurface(
            eglDisplay, configs[0], surface, intArrayOf(EGL14.EGL_NONE), 0
        )
        EGL14.eglMakeCurrent(eglDisplay, eglSurface, eglSurface, eglContext)
    }

    fun stop() { running = false }

    private fun releaseEGL() {
        EGL14.eglMakeCurrent(eglDisplay,

```

```

        EGL14.EGL_NO_SURFACE, EGL14.EGL_NO_SURFACE, EGL14.EGL_NO_CONTEXT)
    EGL14.eglDestroySurface(eglDisplay, eglSurface)
    EGL14.eglDestroyContext(eglDisplay, eglContext)
    EGL14.eglTerminate(eglDisplay)
}
}

```

Render Thread Safety



Never touch EGL from the UI thread

All `EGL14` and `GLES30` calls must happen on the thread where the EGL context was made current (`eglMakeCurrent`). Calling OpenGL functions from the main thread will either silently no-op or crash with `EGL_BAD_ACCESS` . Use a dedicated render thread (shown above) and communicate with it via an `AtomicBoolean` flag or a `Handler` / `HandlerThread` .

TextureView IME Lifecycle

IME views are destroyed and recreated more aggressively than Activity views. The render thread must be started in `onSurfaceTextureAvailable()` and **fully stopped** in `onSurfaceTextureDestroyed()` . Leaking the EGL context across IME show/hide cycles causes GPU memory exhaustion on devices with limited VRAM (≤ 2 GB shared).

```

// Proper lifecycle integration for an IME
override fun onSurfaceTextureDestroyed(st: SurfaceTexture): Boolean {
    backgroundRenderer.stop()
    // Wait for render thread to finish EGL teardown
    backgroundRenderer.awaitTermination(timeout = 500L)
    return true // true = framework releases the SurfaceTexture
}

```

KOTLIN

Further Reading

- [TextureView API Reference \(https://developer.android.com/reference/android/view/TextureView\)](https://developer.android.com/reference/android/view/TextureView) — Official class documentation including lifecycle callbacks.
- [SurfaceTexture & TextureView — Android Graphics Architecture \(https://source.android.com/docs/core/graphics/arch-tv\)](https://source.android.com/docs/core/graphics/arch-tv)
 - Deep dive into how TextureView integrates with the BufferQueue and SurfaceFlinger compositor.

- **OpenGL ES on Android** (<https://developer.android.com/develop/ui/views/graphics/opengl/about-opengl>) —
Official guide covering EGL setup, context management, and GLES API levels.
-

Particle & Ripple Systems ● EXPERT

Per-keystroke particle effects: sparks, confetti, ink splatter, ripples — GPU-driven particle simulation

Particle effects on keystroke are one of the most emotionally engaging customization features available in keyboards. Done well (subtle, fast, appropriate to theme), they delight users. Done poorly (too large, too slow, battery-hungry), they annoy. This section provides complete systems for four particle effects at production quality.

Ripple Effect — Canvas (Beginner Path)

```
data class Ripple(var x: Float, var y: Float, var progress: Float = 0f)

class RippleCanvasOverlay(context: Context) : View(context) {
    private val ripples = mutableListOf<Ripple>()
    private val ripplePaint = Paint(Paint.ANTI_ALIAS_FLAG)
    private var animating = false

    fun spawnRipple(x: Float, y: Float) {
        ripples.add(Ripple(x, y))
        if (!animating) tick()
    }

    private fun tick() {
        animating = true
        postOnAnimation {
            ripples.removeAll { it.progress ≥ 1f }
            ripples.forEach { it.progress += 0.045f }
            invalidate()
            if (ripples.isNotEmpty()) tick() else animating = false
        }
    }

    override fun onDraw(canvas: Canvas) {
        ripples.forEach { ripple →
            val alpha = ((1f - ripple.progress) * 200).toInt()

```

KOTLIN

```

        val radius = ripple.progress * 80f
        ripplePaint.set(rippleColor, alpha, style = Paint.Style.STROKE,
strokeWidth = 2f)
        canvas.drawCircle(ripple.x, ripple.y, radius, ripplePaint)
    }
}
}

```

GPU Particle Spark System (OpenGL ES 3.1)

```

// Compute shader for particle simulation – OpenGL ES 3.1+
// Each particle: [posX, posY, velX, velY, life, maxLife]
#version 310 es
layout(local_size_x = 64) in;

layout(std430, binding = 0) buffer Particles {
    float data[];
};

uniform float uDeltaTime;
uniform float uGravity;

void main() {
    uint i    = gl_GlobalInvocationID.x;
    uint base = i * 6u;

    float life = data[base + 4u];
    if (life ≤ 0.0) return; // Dead particle – skip

    // Integrate position
    data[base + 0u] += data[base + 2u] * uDeltaTime;
    data[base + 1u] += data[base + 3u] * uDeltaTime;

    // Apply gravity and drag
    data[base + 3u] -= uGravity * uDeltaTime;
    data[base + 2u] *= (1.0 - 0.015); // Horizontal drag
    data[base + 3u] *= (1.0 - 0.010); // Vertical drag

    // Decay life
    data[base + 4u] -= uDeltaTime;
}

```

GLSL

**Particle Budget: Maximum 256 active particles at any time**

Testing on Snapdragon 480 (budget 5G): 256 particles with compute shader update + point sprite rendering costs approximately 0.8 ms per frame. 512 particles costs 1.6 ms. The keyboard must keep total frame time under 16 ms while competing with the foreground app. Budget no more than 2 ms total for particle effects.

Custom Layout XML Architecture

● BEGINNER

Designing a flexible, extensible key layout XML schema – beyond the standard Android Keyboard class

Android's built-in `android.inputmethodservice.Keyboard` (<https://developer.android.com/reference/android/inputmethodservice/Keyboard>) XML format (now deprecated) is rigid and doesn't support custom key shapes, arbitrary metadata, or per-key rendering properties. Modern keyboards define their own XML or JSON schema. This section defines a production-quality custom layout schema.

Custom Layout XML Schema

```

<!-- res/xml/layout_qwerty.xml - Custom keyboard layout schema -->
<keyboard
    xmlns:app="http://schemas.android.com/apk/res-auto"
    app:keyHeight="48dp"
    app:keyGap="3dp"
    app:rowGap="2dp"
    app:defaultShape="roundedRect"
    app:defaultDepth="4dp">

    <row>
        <key app:code="q" app:label="Q" app:sublabel="1" />
        <key app:code="w" app:label="W" app:sublabel="2" />
        <!-- ... other keys -->
        <key app:code="p" app:label="P" />
    </row>

    <!-- Action key with custom shape and material override -->
    <key
        app:code="@delete"
        app:iconRes="@drawable/ic_backspace"
        app:widthWeight="1.5"
        app:shape="roundedRect"
        app:material="action_metal"
        app:depth="6dp"

```

XML

```
        app:longPressCode="@selectAll" />

</keyboard>
```

Parsing Custom Layout XML in Kotlin

```
data class KeyDef(
    val code: String,
    val label: String,
    val sublabel: String = "",
    val widthWeight: Float = 1.0f,
    val shape: KeyShape = KeyShape.ROUNDED_RECT,
    val material: String = "standard",
    val depth: Float = 4f, // dp
    val longPressCode: String? = null
)

fun parseKeyboardLayout(context: Context, xmlRes: Int): List<List<KeyDef>> {
    val parser = context.resources.getXml(xmlRes)
    val rows = mutableListOf<MutableList<KeyDef>>()
    var currentRow: MutableList<KeyDef>? = null

    while (parser.next() != XmlPullParser.END_DOCUMENT) {
        if (parser.eventType != XmlPullParser.START_TAG) continue
        when (parser.name) {
            "row" → { currentRow = mutableListOf(); rows.add(currentRow) }
            "key" → currentRow?.add(parseKey(parser))
        }
    }
    return rows
}
```

For full reference implementations of custom layout systems, see the [Android Custom Keyboard Layout guide](https://androiddevpr.com/android-custom-keyboard-layout/) (<https://androiddevpr.com/android-custom-keyboard-layout/>) and the [CodezUp practical example](https://codezup.com/custom-android-keyboard-practical-example/) (<https://codezup.com/custom-android-keyboard-practical-example/>). The [CodezUp IMF guide](https://codezup.com/custom-android-keyboard-input-methods-framework/) (<https://codezup.com/custom-android-keyboard-input-methods-framework/>) provides deep coverage of the Input Methods Framework internals.

Per-Key Shape & Geometry

● INTERMEDIATE

Hexagonal keys, circular keys, organic free-form shapes – custom hit detection and rendering for non-rectangular keys

All standard keyboards use rectangular keys. Breaking this constraint opens design possibilities that are genuinely difficult to achieve elsewhere: **hexagonal honeycomb layouts, circular thumb clusters, organic irregular shapes** for branded experiences. The challenge is that custom shapes require custom hit detection – you can no longer rely on rectangular bounds.

Custom Shape Types

SHAPE	HIT DETECTION	RENDERING TECHNIQUE	USE CASE
Rounded Rect	Rect expand + corner test	Canvas.drawRoundRect / NinePatch	Standard keys, most themes
Hexagonal	Point-in-hexagon test	Canvas path / VBO hex mesh	Gaming keyboards, retro themes
Circular	Distance from center	Canvas.drawCircle / sphere mesh	Thumb keys, emoji picker
Diamond	Rotated rect test	Rotated canvas path	Directional clusters
Free-form SVG	Android Region.contains	PathShape drawable	Branded / novelty keyboards

Hexagonal Key Hit Detection

KOTLIN

```
/**
 * Returns true if (px, py) falls inside a regular hexagon
 * centered at (cx, cy) with circumradius r.
 * Flat-top orientation (rotated 30° from pointy-top).
 */
fun isInsideHexagon(px: Float, py: Float,
                    cx: Float, cy: Float, r: Float): Boolean {
    val dx = Math.abs(px - cx)
    val dy = Math.abs(py - cy)
    // Fast early-out with bounding box
```

```
    if (dx > r || dy > r * 0.866) return false
    // Half-plane test for hex edge
    return (0.866 * dx + 0.5 * dy) ≤ r * 0.866
}

// Honeycomb layout positioning – alternate row offset
fun hexGridCenter(col: Int, row: Int, r: Float): PointF {
    val colSpacing = r * 1.732f // sqrt(3) * r
    val rowSpacing = r * 1.5f
    val offset      = if (row % 2 == 0) 0f else colSpacing * 0.5f
    return PointF(col * colSpacing + offset, row * rowSpacing)
}
```

Dynamic Theme Engine ● INTERMEDIATE

Building a typed, JSON-backed theme system with live preview, Material You integration, and user-editable parameters

A theme engine turns visual customization from a list of settings into a **coherent design system**. The best reference for this is FlorisBoard's Snygg engine (<https://github.com/florisboard/florisboard/tree/main/app/src/main/kotlin/dev/patrickgold/florisboard/snygg>). This section designs a simplified but production-capable alternative using a typed Kotlin data class schema backed by JSON.

Theme Data Model

```

@Serializable
data class KeyboardTheme(
    val id: String,
    val name: String,
    val version: Int = 1,
    val author: String = "",

    // Key surface
    val keyBackground: ColorHex, // e.g. "#2D2A26"
    val keyBorder: ColorHex,
    val keyLabel: ColorHex,
    val keyLabelPressed: ColorHex,
    val keyCornerRadius: Float = 8f,
    val keyDepth: Float = 4f, // 3D depth in dp
    val keyRoughness: Float = 0.4f,
    val keyMetallic: Float = 0.0f,

    // Action keys (Enter, Delete, Shift)
    val actionBackground: ColorHex,
    val actionLabel: ColorHex,

    // Keyboard background
    val boardBackground: ColorHex,
    val backgroundImage: String? = null, // Custom image path
    val backgroundBlur: Float = 0f, // 0-25 blur radius

```

```

    // Particles / effects
    val particlePreset: ParticlePreset = ParticlePreset.NONE,
    val particleColor: ColorHex = "#FFFFFF",

    // Font
    val fontFamily: String = "system", // system | path to custom TTF
    val labelSizeSp: Float = 13f,

    // Material You integration
    val followSystemColors: Boolean = false,
    val iblIntensity: Float = 30_000f
)

enum class ParticlePreset {
    NONE, SPARKS, CONFETTI, RIPPLE, INK_SPLATTER, SNOW, EMBERS
}

```

Material You Integration

```

// Extract Material You dynamic colors and map to keyboard theme
@RequiresApi(31)
fun KeyboardTheme.Companion.fromMaterialYou(context: Context): KeyboardTheme {
    val dynamicColors = DynamicColors.wrapContextIfAvailable(context)
    val typedArray = dynamicColors.obtainStyledAttributes(
        intArrayOf(
            com.google.android.material.R.attr.colorSurface,
            com.google.android.material.R.attr.colorOnSurface,
            com.google.android.material.R.attr.colorPrimary,
            com.google.android.material.R.attr.colorOnPrimary
        )
    )
    return KeyboardTheme(
        id = "material_you_dynamic",
        name = "Material You",
        keyBackground = typedArray.getColorHex(0),
        keyLabel = typedArray.getColorHex(1),
        keyBorder = typedArray.getColorHex(0),
        keyLabelPressed = typedArray.getColorHex(1),
        actionBackground = typedArray.getColorHex(2),
        actionLabel = typedArray.getColorHex(3),
        boardBackground = typedArray.getColorHex(0),
    )
}

```

KOTLIN

```
        followSystemColors = true
    ).also { typedArray.recycle() }
}
```

User-Uploaded Backgrounds ● INTERMEDIATE

Safely loading, resizing, caching, and blurring user-provided images as keyboard backgrounds – with GDPR-conscious storage

Allowing users to set their own photos as keyboard backgrounds is a high-engagement personalization feature. The implementation must handle large images gracefully, apply blur for readability, cache efficiently, and avoid retaining the image after the user uninstalls.

Image Pipeline: Load → Resize → Blur → Cache

```

class BackgroundImagePipeline(
    private val context: Context,
    private val scope: CoroutineScope
) {
    // Target size: keyboard width × height at device density – never larger
    private val targetWidth = context.resources.displayMetrics.widthPixels
    private val targetHeight = (context.resources.displayMetrics.density *
240).toInt()

    suspend fun processUserImage(uri: Uri, blurRadius: Float = 12f): Bitmap? =
        withContext(Dispatchers.IO) {
            try {
                // 1. Decode with inSampleSize to avoid OOM on large photos
                val opts = BitmapFactory.Options().apply {
                    inJustDecodeBounds = true
                    context.contentResolver.openInputStream(uri)?.use {
                        BitmapFactory.decodeStream(it, null, this)
                    }
                    inSampleSize = calculateSampleSize(outWidth, outHeight)
                    inJustDecodeBounds = false
                    inPreferredConfig = Bitmap.Config.ARGB_8888
                }
                val raw = context.contentResolver.openInputStream(uri)?.use {
                    BitmapFactory.decodeStream(it, null, opts)
                } ?: return@withContext null

                // 2. Scale to keyboard dimensions
            }
        }
    }

```

KOTLIN

```

        val scaled = Bitmap.createScaledBitmap(raw, targetWidth, targetHeight, true)
        raw.recycle()

        // 3. RenderScript blur (API 31+ use BlurEffect; older use RS)
        val blurred = applyBlur(scaled, blurRadius)
        scaled.recycle()

        // 4. Cache to internal storage – not accessible outside the app
        cacheBackground(blurred)
        blurred
    } catch (e: Exception) { null }
}

private fun calculateSampleSize(w: Int, h: Int): Int {
    var s = 1
    while (w / s > targetWidth * 2 || h / s > targetHeight * 2) s *= 2
    return s
}
}

```



GDPR & Privacy: User images are sensitive data

Store processed background images in `context.filesDir` (private to your app), never in shared storage. Include a "Clear background" option that deletes the file. Disclose image processing in your Play Store Data Safety declaration. Never upload or transmit the user's image — process entirely on-device.

RenderEffect Blur (API 31+) vs Legacy RenderScript

Android 12 introduced `RenderEffect` as a modern, GPU-accelerated replacement for RenderScript blur. **RenderEffect** (<https://developer.android.com/reference/android/graphics/RenderEffect>) composites on the GPU compositor directly, with no intermediate bitmap allocation.

```

// Modern blur – API 31+ (Android 12)
KOTLIN
fun applyBlur(view: View, radiusPx: Float) {
    if (Build.VERSION.SDK_INT ≥ Build.VERSION_CODES.S) {
        view.setRenderEffect(
            RenderEffect.createBlurEffect(radiusPx, radiusPx,
            Shader.TileMode.CLAMP)
        )
    }
}

```

```

}

// Legacy blur - API 21-30 via Toolkit (RS replacement library)
// Add: implementation("com.google.renderscript:toolkit:latest.release")
fun legacyBlur(source: Bitmap, radius: Int): Bitmap {
    return Toolkit.blur(source, radius)
}

```

Color Extraction for Key Tinting

After loading a user background, extract dominant colors to automatically tint key labels and borders for readability. The Palette library handles this efficiently.

```

// Extract dominant color from background to tint keys
suspend fun extractKeyColors(background: Bitmap): KeyColors =
    withContext(Dispatchers.Default) {
        val palette = Palette.from(background)
            .maximumColorCount(8)
            .generate()

        val dominant = palette.dominantSwatch
        val vibrant   = palette.vibrantSwatch

        KeyColors(
            labelColor = dominant?.bodyTextColor ?: Color.WHITE,
            borderColor = vibrant?.rgb ?: Color.GRAY,
            shadowColor = dominant?.rgb?.let {
                ColorUtils.setAlphaComponent(it, 80)
            } ?: Color.argb(80, 0, 0, 0)
        )
    }

data class KeyColors(
    val labelColor: Int,

```

KOTLIN

```
    val borderColor: Int,  
    val shadowColor: Int,  
)
```

Image Format Handling

FORMAT	SUPPORTED SINCE	NOTES
JPEG	API 1	Best for photo backgrounds; lossy compression keeps file size small.
PNG	API 1	Lossless; preferred for gradient/illustration backgrounds.
WebP (lossy)	API 14	~25% smaller than JPEG at equivalent quality.
WebP (lossless)	API 17	~30% smaller than PNG; good for icon-heavy backgrounds.
HEIF / AVIF	API 26 / 31	Excellent compression; beware patent licensing for HEIF.



Enforce file size limits

Reject images larger than 10 MB before decoding to prevent `OutOfMemoryError` on low-RAM devices. Use `ContentResolver.openAssetFileDescriptor(uri, "r").length` to check file size before calling `BitmapFactory.decodeStream()`. For decoded bitmaps, limit the target size to 2× the keyboard dimensions — anything larger wastes GPU memory after scaling.

Further Reading

- **RenderEffect API Reference** (<https://developer.android.com/reference/android/graphics/RenderEffect>) — GPU-compositor blur, color filter, and shader effects.
- **Photo Picker** (<https://developer.android.com/training/data-storage/shared/photopicker>) — Privacy-friendly image selection without READ_MEDIA permission (API 33+).
- **Palette API Reference** (<https://developer.android.com/reference/androidx/palette/graphics/Palette>) — Color extraction from bitmaps for adaptive UI theming.

Sound & Haptic Packs

● INTERMEDIATE

Designing a theme-based audio and haptic system with user-selectable packs, per-key audio, and `VibrationEffect` composition

Audio and haptics are a critical but often neglected dimension of keyboard personalization. A "mechanical keyboard" theme without clicky audio feedback is incomplete. This section designs a complete sound and haptic pack system where each theme ships its own audio and vibration assets.

Sound Pack Structure

```
// Sound pack descriptor (JSON – shipped in assets/sounds/)
{
  "id":      "mechanical_blue",
  "name":    "Mechanical Blue Switch",
  "version": 1,
  "samples": {
    "key_press":  "samples/blue_press.ogg",
    "key_release": "samples/blue_release.ogg",
    "space_press": "samples/blue_space.ogg",
    "delete_press": "samples/blue_delete.ogg",
    "return_press": "samples/blue_return.ogg"
  },
  "volume":    0.65,
  "pitchMin":  0.95,
  "pitchMax":  1.05
}
```

JSON

SoundPool-Based Playback Engine

```
class KeySoundEngine(context: Context) {

    private val soundPool = SoundPool.Builder()
        .setMaxStreams(4)
        .setAudioAttributes(
            AudioAttributes.Builder()
```

KOTLIN

```

        .setUsage(AudioAttributes.USAGE_ASSISTANCE_SONIFICATION)
        .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
        .build()
    ).build()

private val soundIds = mutableMapOf<String, Int>()

fun loadPack(context: Context, pack: SoundPack) {
    soundIds.clear()
    pack.samples.forEach { (key, assetPath) →
        context.assets.openFd(assetPath).use { fd →
            soundIds[key] = soundPool.load(fd, 1)
        }
    }
}

fun play(eventType: String, pack: SoundPack) {
    val id = soundIds[eventType] ?: soundIds["key_press"] ?: return
    val pitch = Random.nextFloat() * (pack.pitchMax - pack.pitchMin) +
pack.pitchMin
    soundPool.play(id, pack.volume, pack.volume, 0, 0, pitch)
}

fun release() { soundPool.release() }
}

// VibrationEffect predefined effects – API 29+
fun buildKeyHaptic(preset: HapticPreset): VibrationEffect = when (preset) {
    HapticPreset.CLICK →
        VibrationEffect.createPredefined(VibrationEffect.EFFECT_CLICK)
    HapticPreset.TICK →
        VibrationEffect.createPredefined(VibrationEffect.EFFECT_TICK)
    HapticPreset.HEAVY →
        VibrationEffect.createPredefined(VibrationEffect.EFFECT_HEAVY_CLICK)
    HapticPreset.DOUBLE_CLICK →
        VibrationEffect.createPredefined(VibrationEffect.EFFECT_DOUBLE_CLICK)
    HapticPreset.CUSTOM → VibrationEffect.createWaveform(
        longArrayOf(0, 15, 30, 15), // timings ms
        intArrayOf(0, 200, 0, 100), // amplitudes
        -1 // no repeat
    )
}

```

AudioFocus & Low-Latency Playback

Keyboard sounds must never steal audio focus from music or calls. Request

`AUDIOFOCUS_GAIN_TRANSIENT_MAY_DUCK` only if you play continuous audio (rare); for short key-click samples, skip focus entirely and use the **sonification** usage type so the system mixes your clips automatically.

```
object KeySoundPolicy {

    /**
     * Returns AudioAttributes tuned for minimal-latency key feedback.
     * USAGE_ASSISTANCE_SONIFICATION tells the OS this is UI feedback,
     * not media – so it ducks under music instead of pausing it.
     */
    fun feedbackAttributes(): AudioAttributes =
        AudioAttributes.Builder()
            .setUsage(AudioAttributes.USAGE_ASSISTANCE_SONIFICATION)
            .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
            .build()

    /**
     * Pre-warm the SoundPool by loading all samples BEFORE the first
     * keystroke. SoundPool.load() is asynchronous – track readiness
     * via OnLoadCompleteListener and block playback until all IDs
     * report success.
     */
    fun buildOptimizedPool(maxStreams: Int = 6): SoundPool =
        SoundPool.Builder()
            .setMaxStreams(maxStreams)
            .setAudioAttributes(feedbackAttributes())
            .build()
}
```



Latency tip

Keep samples under 100 ms and 48 kHz / 16-bit mono. Shorter files load faster into the SoundPool buffer and avoid audible scheduling jitter on low-end chipsets. OGG Vorbis gives the best size-to-quality ratio; Opus is smaller but requires API 29+.

FORMAT	CODEC	MIN API	TYPICAL LATENCY	NOTES
<code>.ogg</code>	Vorbis	1	5–10 ms	Best compatibility; preferred for key clicks
<code>.opus</code>	Opus	29	5–8 ms	Smaller files; ideal when min SDK ≥ 29
<code>.wav</code>	PCM	1	2–5 ms	Lowest latency but 10× file size
<code>.mp3</code>	MP3	1	15–30 ms	Avoid — decoder latency too high for feedback
<code>.flac</code>	FLAC	14	8–15 ms	Lossless; overkill for short UI sounds

Rich Haptics with `VibrationEffect.Composition`

API 31 introduced `VibrationEffect.startComposition()`, which lets you chain haptic primitives with per-primitive intensity scaling and inter-primitive delays. This replaces the older waveform API with a richer vocabulary of tactile sensations that the HAL can optimize for specific actuator hardware.

```
// Requires API 31+ (Android 12)
import android.os.VibrationEffect
import android.os.VibrationEffect.Composition

object RichHaptics {

    /** Crisp single-tap – ideal for standard key presses. */
    fun keyTap(): VibrationEffect =
        VibrationEffect.startComposition()
            .addPrimitive(Composition.PRIMITIVE_CLICK, 0.6f)
            .compose()

    /** Heavier click for space bar or return key. */
    fun actionTap(): VibrationEffect =
        VibrationEffect.startComposition()
            .addPrimitive(Composition.PRIMITIVE_THUD, 0.8f)
            .compose()

    /** Soft tick for swipe-gesture waypoints (each key crossed). */
    fun gestureTick(): VibrationEffect =
        VibrationEffect.startComposition()
            .addPrimitive(Composition.PRIMITIVE_TICK, 0.3f)
            .compose()

    /** Delete hold – rising buzz every 50 ms while backspace repeats. */
    fun deleteRepeat(count: Int): VibrationEffect {
        val comp = VibrationEffect.startComposition()
        for (i in 0 until count.coerceAtMost(20)) {

```

```

        val scale = (0.2f + i * 0.04f).coerceAtMost(1.0f)
        comp.addPrimitive(Composition.PRIMITIVE_TICK, scale, 50)
    }
    return comp.compose()
}

/** Long-press pop-up – slow rise then sharp click. */
fun longPressReveal(): VibrationEffect =
    VibrationEffect.startComposition()
        .addPrimitive(Composition.PRIMITIVE_SLOW_RISE, 0.5f)
        .addPrimitive(Composition.PRIMITIVE_CLICK, 0.9f, 30)
        .compose()
}

// Map key roles to haptic patterns
fun hapticForKey(code: Int): VibrationEffect = when (code) {
    KeyEvent.KEYCODE_DEL → RichHaptics.deleteRepeat(1)
    KeyEvent.KEYCODE_ENTER,
    KeyEvent.KEYCODE_SPACE → RichHaptics.actionTap()
    else → RichHaptics.keyTap()
}

```



Primitive availability

`Vibrator.arePrimitivesSupported()` returns a boolean array indicating which primitives the device actuator actually handles. Always query before composing; fall back to `createPredefined(EFFECT_CLICK)` on devices that lack composition support.

Per-Key Haptic Variation Map

Rather than one-haptic-fits-all, map each key role to a distinct tactile signature. Users subconsciously learn the difference between a letter tick, a modifier thud, and a delete buzz — making the keyboard feel more responsive without visual confirmation.

KEY ROLE	PRIMITIVE (API 31+)	FALLBACK (API 26+)	INTENSITY	UX RATIONALE
Letter	PRIMITIVE_TICK	EFFECT_TICK	0.4	Light & fast — no fatigue over long typing
Space	PRIMITIVE_CLICK	EFFECT_CLICK	0.6	Word-boundary confirmation
Enter / Return	PRIMITIVE_THUD	EFFECT_HEAVY_CLICK K	0.8	Action commitment signal
Backspace	PRIMITIVE_TICK (escalating)	EFFECT_TICK	0.3 → 1.0	Rising intensity during hold-to-delete
Shift / Caps	PRIMITIVE_CLICK	EFFECT_CLICK	0.7	Mode-change acknowledgment
Symbol toggle	PRIMITIVE_SLOW_RISE	EFFECT_TICK	0.5	Layer-switch feedback
Long-press popup	SLOW_RISE + CLICK	EFFECT_DOUBLE_CLICK K	0.5 + 0.9	Build-up then pop — matches visual animation

Sound Pack Download & Management

Shipping many packs inside the APK bloats download size. A better pattern is to bundle one default pack and download extras on demand. Store packs in app-private internal storage and validate the JSON manifest before loading.

KOTLIN

```

data class SoundPackMeta(
    val id: String,
    val displayName: String,
    val version: Int,
    val sizeBytes: Long,
    val downloadUrl: String,
    val sha256: String,      // integrity check after download
)

class SoundPackManager(
    private val context: Context,
    private val engine: KeySoundEngine,
) {
    private val packDir = File(context.filesDir, "sound_packs")

    /** List locally available packs. */
    fun installedPacks(): List<String> =
        packDir.listFiles { f → f.isDirectory }
            ?.map { it.name }

```

```

        ?: emptyList()

    /** Load and activate a pack by ID. */
    fun activate(packId: String) {
        val manifestFile = File(packDir, "$packId/manifest.json")
        require(manifestFile.exists()) { "Pack not installed: $packId" }

        val pack = Json.decodeFromString<SoundPack>(manifestFile.readText())
        engine.loadPack(context, pack, baseDir = File(packDir, packId))
    }

    /** Delete a downloaded pack. */
    fun remove(packId: String) {
        File(packDir, packId).deleteRecursively()
    }
}

```



Integrity verification

Always hash downloaded ZIP files with `MessageDigest.getInstance("SHA-256")` and compare against `SoundPackMeta.sha256` before extracting. This prevents tampered packs from reaching the playback engine.

Coordinating Sound and Haptics

Sound and vibration should fire at the exact same instant. `SoundPool.play()` is effectively synchronous after load, but `Vibrator.vibrate()` may queue behind other effects. The safest pattern is to fire both from the same `onPress()` callback on the UI thread — never from a background coroutine.

```

class FeedbackCoordinator(
    private val soundEngine: KeySoundEngine,
    private val vibrator: Vibrator,
    private val preferences: KeyboardPreferences,
) {
    /** Call from onPress() on the main thread. */
    fun onKeyDown(code: Int, pack: SoundPack?) {
        if (preferences.soundEnabled && pack != null) {
            soundEngine.play(eventTypeFor(code), pack)
        }
        if (preferences.hapticEnabled) {
            vibrator.vibrate(hapticForKey(code))
        }
    }
}

```

KOTLIN

```

    }
}

private fun eventTypeFor(code: Int): String = when (code) {
    KeyEvent.KEYCODE_SPACE → "space_press"
    KeyEvent.KEYCODE_DEL   → "delete_press"
    KeyEvent.KEYCODE_ENTER → "return_press"
    else                    → "key_press"
}
}

```

External Resources

- **VibrationEffect API Reference** (<https://developer.android.com/reference/android/os/VibrationEffect>) — full list of predefined effects and composition primitives.
- **SoundPool API Reference** (<https://developer.android.com/reference/android/media/SoundPool>) — stream management, priority, and load callbacks.
- **Custom Haptic Effects Guide** (<https://developer.android.com/develop/ui/views/haptics/custom-haptic-effects>) — official walkthrough of Composition API and `arePrimitivesSupported()`.
- **AudioAttributes Reference** (<https://developer.android.com/reference/android/media/AudioAttributes>) — usage types, content types, and how the mixer prioritizes sonification streams.

Font & Label Customization

● INTERMEDIATE

Custom typefaces on key labels – loading TTF/OTF fonts, variable fonts, icon fonts, and rendering at multiple scales

Key labels are the most legible text on a keyboard. Changing the typeface dramatically alters personality: a geometric sans-serif feels modern and technical; a rounded display font feels friendly; a monospaced font signals developer-centric branding. This section covers the full pipeline from font asset to rendered label.

Custom Font Pipeline

```
object KeyFontCache {
    private val cache = mutableMapOf<String, Typeface>()

    fun load(context: Context, fontIdentifier: String): Typeface =
        cache.getOrPut(fontIdentifier) {
            when {
                fontIdentifier == "system" → Typeface.DEFAULT
                fontIdentifier.startsWith("assets:") → {
                    val path = fontIdentifier.removePrefix("assets:")
                    Typeface.createFromAsset(context.assets, path)
                }
                fontIdentifier.startsWith("file:") → {
                    val path = fontIdentifier.removePrefix("file:")
                    Typeface.createFromFile(File(path))
                }
                else → Typeface.DEFAULT
            }
        }
}

// Variable font support (API 26+) – adjust weight per theme
fun loadVariableFont(context: Context, assetPath: String, weight: Int = 500):
Typeface {
    val builder = Typeface.Builder(context.assets, assetPath)
```

KOTLIN

```
builder.setFontVariationSettings("'wght' $weight")
return builder.build()
}
```



Use variable fonts for compact size + maximum flexibility

A single variable font file (e.g., Google's Recursive, Roboto Flex) covers the full weight and width axes — replacing 8+ static font files. For keyboards shipping multiple theme packs, this cuts asset size significantly. Variable fonts are supported from API 26 via

```
Typeface.Builder.setFontVariationSettings()
```

Text Measurement & Label Fitting

Key labels must be centered both horizontally and vertically inside an arbitrarily sized cap.

Use `Paint.getTextBounds()` for bounding-box metrics and `Paint.measureText()` for advance-width. The difference matters: `getTextBounds()` returns ink bounds (tight around glyphs), while `measureText()` returns the logical advance including side bearings.

```
object LabelRenderer {

    private val bounds = Rect()

    /**
     * Draws [label] centred inside a key whose origin is (keyX, keyY)
     * and whose dimensions are (keyW, keyH) in pixels.
     * Automatically scales the font down if the label overflows.
     */
    fun drawCentred(
        canvas: Canvas,
        label: String,
        keyX: Float, keyY: Float,
        keyW: Float, keyH: Float,
        paint: Paint,
        maxTextSizeSp: Float = 22f,
    ) {
        paint.textSize = maxTextSizeSp
        paint.getTextBounds(label, 0, label.length, bounds)

        // Shrink if label overflows 80 % of key width
        val maxW = keyW * 0.8f
        if (bounds.width() > maxW) {
            paint.textSize *= maxW / bounds.width()
        }
    }
}
```

KOTLIN

```

        paint.getTextBounds(label, 0, label.length, bounds)
    }

    val x = keyX + (keyW - bounds.width()) / 2f - bounds.left
    val y = keyY + (keyH + bounds.height()) / 2f
    canvas.drawText(label, x, y, paint)
}
}

```

Emoji Rendering Pipeline

Emoji glyph coverage varies wildly across vendor fonts. **EmojiCompat** from

`androidx.emoji2` patches missing glyphs at runtime by replacing codepoints with `EmojiSpan` bitmaps from a downloadable font. Initialize it early — ideally in `Application.onCreate()` — so the library is ready before the first emoji draw call.

```

import androidx.emoji2.text.EmojiCompat
import androidx.emoji2.bundled.BundledEmojiCompatConfig
                                                                    KOTLIN

// In Application.onCreate()
val config = BundledEmojiCompatConfig(this)
    .setReplaceAll(true)           // override vendor fonts
EmojiCompat.init(config)

// Alternatively: downloadable config (smaller APK, network needed)
val fontRequest = FontRequest(
    "com.google.android.gms.fonts",
    "com.google.android.gms",
    "Noto Color Emoji Compat",
    R.array.com_google_android_gms_fonts_certs
)
val dlConfig = FontRequestEmojiCompatConfig(this, fontRequest)
EmojiCompat.init(dlConfig)

```



Canvas emoji rendering

`EmojiCompat` works transparently with `Canvas.drawText()` as long as you call `EmojiCompat.get().process(text)` before passing the label to the paint. For custom `Views` that bypass `TextView`, this is the required integration step.

Icon Fonts for Special Keys

Instead of shipping raster PNGs for backspace arrows, shift icons, or globe symbols, load an icon font. Google's **Material Symbols** variable font supports *fill*, *weight*, *grade*, and *optical size* axes — giving you crisp vector icons at any density without per-DPI assets.

```
object IconFont {  
    private var typeface: Typeface? = null  
  
    fun get(context: Context): Typeface {  
        if (typeface == null) {  
            typeface = Typeface.Builder(context.assets,  
                "fonts/MaterialSymbolsRounded.ttf")  
                .setFontVariationSettings("'FILL' 1, 'wght' 400, 'GRAD' 0,  
                'opsz' 24")  
                .build()  
        }  
        return typeface!!  
    }  
  
    // Material Symbols codepoints for common keyboard icons  
    const val BACKSPACE = "\uE14A" // backspace  
    const val SHIFT     = "\uE318" // keyboard_capslock  
    const val GLOBE     = "\uE894" // language  
    const val MIC       = "\uE029" // mic  
    const val RETURN    = "\uE31B" // keyboard_return  
    const val SETTINGS  = "\uE8B8" // settings  
    const val EMOJI     = "\uE24E" // mood  
}
```

KOTLIN

Multi-Script Text Rendering

Keyboards that support multiple scripts (Latin, Cyrillic, Arabic, Devanagari, CJK) must handle right-to-left layout, complex shaping, and fallback font chains. Android's

`Typeface.Builder` supports a `setFallback()` parameter from API 29. For older APIs, use

`Paint.setTextLocale()` to hint the shaper.

SCRIPT	DIRECTION	SHAPING	RECOMMENDED FONT	NOTES
Latin	LTR	Simple	Roboto Flex	Variable; covers extended Latin
Cyrillic	LTR	Simple	Roboto Flex	Same font, different range
Arabic	RTL	Complex (joining)	Noto Sans Arabic	Mirror left/right key layout
Devanagari	LTR	Complex (conjuncts)	Noto Sans Devanagari	Ligatures require HarfBuzz shaping
CJK	LTR / TTB	Simple (no joining)	Noto Sans CJK	Large file — subset or download on demand
Thai	LTR	Complex (tonal marks above)	Noto Sans Thai	Above-base marks need extra vertical space



Font subsetting for APK size

A full Noto Sans CJK font is 15+ MB. Use `pyftsubset` (from `fonttools`) to strip unused glyphs, or use the [Downloadable Fonts API](https://developer.android.com/develop/ui/views/text-and-emoji/downloadable-fonts) (<https://developer.android.com/develop/ui/views/text-and-emoji/downloadable-fonts>) to fetch fonts from Google Fonts at runtime — zero APK cost.

Downloadable Fonts API

```
import androidx.core.provider.FontRequest
import androidx.core.provider.FontsContractCompat

fun requestFont(context: Context, fontFamily: String, callback: (Typeface) → Unit) {
    val request = FontRequest(
        "com.google.android.gms.fonts",
        "com.google.android.gms",
        fontFamily,
        R.array.com_google_android_gms_fonts_certs
    )
    FontsContractCompat.requestFont(
        context,
        request,
        object : FontsContractCompat.FontRequestCallback() {
            override fun onTypefaceRetrieved(typeface: Typeface) {
                callback(typeface)
            }
            override fun onTypefaceRequestFailed(reason: Int) {
                // Handle failure
            }
        }
    )
}
```

KOTLIN

```
        callback(Typeface.DEFAULT) // graceful fallback
    }
},
Handler(Looper.getMainLooper())
)
}
```

External Resources

- **Downloadable Fonts Guide** (<https://developer.android.com/develop/ui/views/text-and-emoji/downloadable-fonts>)
 - Google Fonts provider integration and cache behavior.
 - **EmojiCompat Library** (<https://developer.android.com/develop/ui/views/text-and-emoji/emoji-compat>) — bundled vs downloadable config, version support table.
 - **Material Symbols — Google Fonts** (<https://fonts.google.com/icons>) — searchable icon catalog with variable-font axis controls.
 - **Typeface.Builder API Reference** (<https://developer.android.com/reference/android/graphics/Typeface.Builder>)
 - variable font axes, font variation settings, and fallback chains.
-

Unity Android Plugin ● EXPERT

Embedding a Unity scene as the keyboard's rendering layer — plugin lifecycle, UnityPlayer integration, and input routing

For teams with existing Unity expertise and 3D assets, embedding Unity as the keyboard renderer provides access to the full Unity asset ecosystem — particle systems, shader graphs, post-processing, and asset store content. The integration requires a Unity Android plugin (AAR) and careful lifecycle bridging between the IME and `UnityPlayer`.



Unity in an IME: Significant caveats

Unity's `UnityPlayer` is designed to be the primary activity renderer — not a background service renderer. It assumes it can take full control of the Android Activity, GL context, and main loop. Embedding it in an IME service requires hacking around these assumptions and is **unsupported by Unity Technologies**

. The approach described here is an advanced workaround, not an official pattern. Budget 4–6 weeks for integration work, and test exhaustively across OEMs. Unity's standard LTS (2022/2023) is the only version worth attempting this with.

Unity as Android Library Module

```
// build.gradle.kts (IME module)
// Unity exports its runtime as an AAR — add it as a local dependency
dependencies {
    implementation(fileTree(dir = "libs", include = listOf("*.aar")))
    // The unity-classes.jar ships inside the Unity Android export
}
```

GRADLE

```
// In proguard-rules.pro – keep Unity classes
// -keep class com.unity3d.** { *; }
// -keep class com.unity.androidnotifications.** { *; }
```

UnityPlayer Bridge in IME Service

```
class UnityKeyboardService : InputMethodService() {
    private var unityPlayer: UnityPlayer? = null

    override fun onCreate() {
        super.onCreate()
        // UnityPlayer needs an Activity context for its initialization,
        // but IME has only a Service context. We pass 'this' and accept
        // that some Unity features (e.g., AR) will not work.
        try {
            unityPlayer = UnityPlayer(this, null)
        } catch (e: Exception) {
            // Fallback to standard Canvas rendering
            Log.e("Unity", "UnityPlayer init failed, falling back", e)
        }
    }

    override fun onCreateInputView(): View {
        val up = unityPlayer ?: return buildFallbackView()
        up.requestFocus()
        // Send Unity a message to set up keyboard scene
        UnityPlayer.UnitySendMessage("KeyboardManager", "SetupScene", "")
        return up
    }

    // Route key taps to Unity – Unity renders press animation
    fun onKeyPressed(keyCode: String) {
        UnityPlayer.UnitySendMessage("KeyboardManager", "OnKeyPress", keyCode)
        currentInputConnection?.commitText(keyCode, 1)
    }

    override fun onStartInputView(info: EditorInfo, restarting: Boolean) {
        super.onStartInputView(info, restarting)
        unityPlayer?.resume()
    }

    override fun onFinishInputView(finishingInput: Boolean) {
        unityPlayer?.pause()
    }
}
```

KOTLIN

```
        super.onFinishInputView(finishingInput)
    }

    override fun onDestroy() {
        unityPlayer?.destroy()
        super.onDestroy()
    }
}
```

Godot Android Library ● EXPERT

Using Godot's Android library (GodotFragment / GodotHost) to render a keyboard scene, Godot-Android signal bridge, and input routing

Godot 4.x provides an official Android library (https://docs.godotengine.org/en/stable/tutorials/platform/android/android_library.html) that allows embedding a Godot scene inside an Android application. Unlike Unity, Godot was designed from the start to support being embedded as a library — making it a more practical choice for keyboard rendering than Unity.

Setup: Godot Android Library

```
// 1. Export your Godot keyboard scene as an Android library
//    In Godot: Project → Export → Android → Export as Library
//    This generates a .aar file

// 2. Add to your IME project
// settings.gradle.kts
dependencyResolutionManagement {
    repositories {
        maven { url = uri("https://godot.im/releases") }
    }
}

// build.gradle.kts
dependencies {
    implementation("org.godotengine:godot:4.3.0.stable")
    implementation(files("libs/keyboard_scene.aar"))
}
```

GRADLE

GodotFragment in IME

```
class GodotKeyboardService : InputMethodService(), GodotHost {

    private var godotFragment: GodotFragment? = null
```

KOTLIN

```

override fun onCreateInputView(): View {
    // Godot requires a FragmentActivity context for GodotFragment.
    // For an IME, we wrap in a custom FrameLayout that hosts the fragment.
    val container = FrameLayout(this)

    godotFragment = GodotFragment().also { fragment →
        // GodotFragment embeds directly as a View in API 28+
        // Pass startup parameters to the Godot scene
        fragment.arguments = Bundle().apply {
            putStringArray("command_line", arrayOf(
                "--main-pack", "keyboard_scene.pck"
            ))
        }
        container.addView(fragment.requireView(),
            FrameLayout.LayoutParams.MATCH_PARENT,
            FrameLayout.LayoutParams.MATCH_PARENT)
    }
    return container
}

// Send key events to Godot scene via GDScript bridge
fun onKeyTapped(keyLabel: String) {
    godotFragment?.godot?.runOnGodotThread {
        // Call a GDScript function via Android Java/Kotlin bridge
        // In Godot scene: keyboard_bridge.gd defines on_key_pressed(label)
        GodotLib.callDeferred("keyboard_bridge", "on_key_pressed",
            arrayOf(keyLabel))
    }
    currentInputConnection?.commitText(keyLabel, 1)
}

override fun getActivity(): FragmentActivity? = null // IME has no Activity
override fun getHostPlugins(godot: Godot) = setOf<GodotPlugin>()
}

```

GDScript Keyboard Scene

The Godot scene defines the visual layout as a node tree. Each key is a `Button` or `Area3D` node. A GDScript singleton (`keyboard_bridge.gd`) receives tap events from the Android host and dispatches visual feedback — press animations, particle bursts, or shader transitions.

```
# keyboard_bridge.gd - autoloaded singleton
extends Node

## Emitted when the Android host taps a key.
signal key_pressed(label: String)
signal key_released(label: String)

## Called from Kotlin via GodotLib.calldeferred().
func on_key_pressed(label: String) → void:
    key_pressed.emit(label)

func on_key_released(label: String) → void:
    key_released.emit(label)
```

```
# key_cap.gd - attach to each key MeshInstance3D
extends MeshInstance3D

@export var label: String = "A"
@export var press_depth: float = 0.03

var tween: Tween

func _ready() → void:
    KeyboardBridge.key_pressed.connect(_on_pressed)
    KeyboardBridge.key_released.connect(_on_released)

func _on_pressed(pressed_label: String) → void:
    if pressed_label ≠ label:
        return
    if tween:
        tween.kill()
    tween = create_tween().set_ease(Tween.EASE_OUT).set_trans(Tween.TRANS_CUBIC)
    tween.tween_property(self, "position:y", position.y - press_depth, 0.06)

func _on_released(released_label: String) → void:
    if released_label ≠ label:
        return
    if tween:
        tween.kill()
```

```
tween =
create_tween().set_ease(Tween.EASE_OUT).set_trans(Tween.TRANS_BOUNCE)
tween.tween_property(self, "position:y", 0.0, 0.15)
```

Signal Bridge: Android ↔ Godot

Communication between the Android host and the Godot runtime crosses a JNI boundary. The safe pattern is:

1. **Android → Godot:** `GodotLib.callDeferred(nodePath, method, args)` — queues a call on the Godot main thread. Safe from any Android thread.
2. **Godot → Android:** Register a `GodotPlugin` that exposes `@UsedByGodot`-annotated methods. GDScript calls them via `Engine.get_singleton("PluginName").method()`.

```
class KeyboardPlugin(godot: Godot) : GodotPlugin(godot) {
    override fun getPluginName() = "KeyboardPlugin"

    /** Called from GDScript to commit text to the active input field. */
    @UsedByGodot
    fun commitText(text: String) {
        // Post to main thread because IME calls must happen there
        activity?.runOnUiThread {
            // Access the InputMethodService's currentInputConnection
            GodotKeyboardService.instance
                ?.currentInputConnection
                ?.commitText(text, 1)
        }
    }

    @UsedByGodot
    fun sendKeyEvent(keyCode: Int) {
        activity?.runOnUiThread {
            GodotKeyboardService.instance
                ?.currentInputConnection
                ?.sendKeyEvent(KeyEvent(KeyEvent.ACTION_DOWN, keyCode))
        }
    }
}
```

```
        }  
    }  
}
```

3D Engine Comparison for Keyboard Rendering

CRITERIA	GODOT 4.X	UNITY (AS LIBRARY)	NATIVE OPENGL ES / VULKAN
Library size (AAR/SO)	~12 MB	~30–50 MB	< 1 MB
Startup time	200–400 ms	500–1200 ms	< 50 ms
Render API	Vulkan / GL ES 3.0	Vulkan / GL ES 3.0	Direct GL / Vulkan
Scene editor	✓ Full editor	✓ Full editor	✗ Code-only
Hot reload	✓ GDScript	⚠ Limited on device	✗
License	MIT	Proprietary (free tier)	N/A
Battery impact (idle)	Medium — needs <code>set_process(false)</code>	High — Unity player loop	Low — manual draw loop



Battery & thermal

Godot's main loop runs at 60 fps by default, even when idle. Set `Engine.max_fps = 0` and use `set_process(false)` on all nodes when the keyboard is not visible. Resume rendering only when the IME window opens. Without this, background GPU work will drain 5–10 % battery per hour.

Asset Pipeline: Editor to APK

1. Design the keyboard scene in Godot Editor (3D or 2D).
2. Export as `.pck` (Godot pack file) — not a standalone APK.
3. Place the `.pck` in `assets/` of the Android project.
4. The `GodotFragment` loads the pack at runtime via the `--main-pack` command-line argument.

5. For multiple themes, export separate packs and swap them at runtime by reinitializing the fragment.

External Resources

- **Godot as Android Library** (https://docs.godotengine.org/en/stable/tutorials/platform/android/android_library.html)
— official guide for embedding Godot in an existing Android app.
 - **Godot Android Plugin System** (https://docs.godotengine.org/en/stable/tutorials/platform/android/android_plugin.html) — creating `GodotPlugin` subclasses with `@UsedByGodot` annotation.
 - **Tween API Reference** (https://docs.godotengine.org/en/stable/classes/class_tween.html) — ease curves, chaining, and property animation.
-

SECTION 25

AR Keyboard with Sceneform

● EXPERT

Augmented Reality keys that float in the real world — Sceneform/ARCore setup, plane detection, and overlay routing for IME

An AR keyboard projects 3D keys into the camera feed, making them appear to rest on surfaces in the real world. This is an advanced novelty feature — not appropriate as a default experience — but an impressive demonstration of what's possible with the Sceneform + ARCore stack integrated into an IME.



AR keyboard is a showcase feature, not a usability feature

AR requires the camera, which drains battery rapidly, requires the `CAMERA` permission (showing a dangerous permission prompt), and makes typing significantly harder. Gate the AR mode behind a clear opt-in and provide an immediate exit. Only enable it in apps where the camera context makes sense (AR apps, games, novelty experiences).

Sceneform AR Overlay Architecture

```
// Gradle dependency
implementation("io.github.sceneview:arsceneview:2.2.1")

// AndroidManifest.xml additions for AR keyboard mode
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera.ar" android:required="false" />
<meta-data android:name="com.google.ar.core" android:value="optional" />
```

GRADLE

```
class ARKeyboardService : InputMethodService() {

    private var arMode = false

    override fun onCreateInputView(): View {
        // Check ARCore availability before attempting AR
        arMode = ArCoreApk.getInstance().checkAvailability(this) ==
            ArCoreApk.Availability.SUPPORTED_INSTALLED
    }
}
```

KOTLIN

```

        && checkSelfPermission(Manifest.permission.CAMERA) ==
            PackageManager.PERMISSION_GRANTED

    return if (arMode) buildARView() else buildStandardView()
}

private fun buildARView(): View {
    // ARSceneView handles camera, plane detection, and rendering
    return ARSceneView(this).apply {
        planeRenderer.isEnabled = true
        // Place keyboard anchor on detected horizontal plane
        onSessionUpdated = { session, frame →
            if (getAnchors().isEmpty()) {
                frame.getUpdatedTrackables(Plane::class.java)
                    .firstOrNull { it.trackingState ==
TrackingState.TRACKING
                                && it.type ==
Plane.Type.HORIZONTAL_UPWARD_FACING }
                    ?.let { plane →
                        addChildNode(buildKeyboardNode(plane))
                    }
            }
        }
    }
}

```

Key Node Placement & Grid Layout

Once a horizontal plane is detected, individual key nodes are placed in a 3×10 grid anchored to the plane's center. Each key is a `ModelNode` loaded from a `.glb` asset, scaled to approximate physical key size in AR space (≈ 16 mm square at arm's length).

```

private fun buildKeyboardNode(plane: Plane): AnchorNode {
    val pose = plane.centerPose
    val anchor = plane.createAnchor(pose)
    val root = AnchorNode(anchor)

    val rows = listOf(
        "QWERTYUIOP",
        "ASDFGHJKL",
        "ZXCVBNM"
    )
    val keySpacing = 0.018f // 18 mm centres

```

KOTLIN

```

rows.forEachIndexed { row, letters →
    val offsetX = (rows[0].length - letters.length) * keySpacing / 2f
    letters.forEachIndexed { col, char →
        val node = createKeyNode(char.toString())
        node.localPosition = Vector3(
            offsetX + col * keySpacing,
            0f,
            row * keySpacing
        )
        root.addChildNode(node)
    }
}
activeAnchors.add(anchor)
return root
}

private fun createKeyNode(label: String): ModelNode {
    return ModelNode().apply {
        loadModelGlbAsync(
            glbFileLocation = "models/ar_key_cap.glb",
            autoAnimate      = false
        )
        localScale = Vector3(0.016f, 0.016f, 0.016f)
        name = label
    }
}
}

```



GLB Asset Pipeline

Export key cap models from Blender with

Draco

mesh compression enabled (`glTF 2.0 → .glb` , compression level 6). A full QWERTY grid of 26 identical caps with instanced rendering consumes ≈ 3 MB GPU memory — well within AR-capable device budgets. Place `.glb` files in `assets/models/` .

Touch-to-Key Raycasting

Screen-space touch coordinates are projected into the AR scene using `Frame.hitTest()` .

The closest [HitResult](https://developers.google.com/ar/reference/java/arcore/reference/com/google/ar/core/HitResult) [↗] (<https://developers.google.com/ar/reference/java/arcore/reference/com/google/ar/core/HitResult>) whose trackable is the keyboard anchor plane identifies which key node the user tapped.

```

private fun handleARTouch(
    frame: Frame,
    motionEvent: MotionEvent,
    keyboardRoot: AnchorNode
): String? {
    val hits = frame.hitTest(motionEvent)
    val planeHit = hits.firstOrNull { hit →
        hit.trackable is Plane &&
        (hit.trackable as Plane).isPoseInPolygon(hit.hitPose)
    } ?: return null

    // Convert hit pose to keyboard-local coordinates
    val worldPos = planeHit.hitPose.translation
    val anchorInv = keyboardRoot.anchor!!.pose.inverse()
    val local = anchorInv.compose(Pose(worldPos, floatArrayOf(0f, 0f, 0f, 1f)))

    // Find nearest key node by XZ distance
    var closest: ModelNode? = null
    var minDist = Float.MAX_VALUE
    for (child in keyboardRoot.childNodes.filterIsInstance<ModelNode>()) {
        val dx = child.localPosition.x - local.tx()
        val dz = child.localPosition.z - local.tz()
        val d = dx * dx + dz * dz
        if (d < minDist) { minDist = d; closest = child }
    }

    return closest?.name // The label character
}

```

Anchor Lifecycle Management

AR anchors consume ARCore tracking resources and must be explicitly detached when the keyboard exits AR mode or the IME is destroyed. Leaking anchors degrades tracking quality and increases battery drain. The lifecycle wrapper below ties anchor cleanup to the service lifecycle.

```

class AnchorManager {
    private val anchors = mutableListOf<Anchor>()

    fun track(anchor: Anchor) { anchors.add(anchor) }

    fun detachAll() {
        anchors.forEach { if (it.trackingState != TrackingState.STOPPED) it.de-

```

```
tach() }  
    anchors.clear()  
}  
  
fun pruneStale() {  
    anchors.removeAll { it.trackingState == TrackingState.STOPPED }  
}  
}  
  
// In ARKeyboardService:  
private val anchorManager = AnchorManager()  
  
override fun onDestroy() {  
    anchorManager.detachAll()  
    super.onDestroy()  
}
```



Battery & Thermal Impact

AR keyboard mode keeps the camera sensor, GPU, and ARCore pose estimation running simultaneously. On mid-range devices this draws

2–4 W

additional power and can trigger thermal throttling within 3–5 minutes. Implement an automatic timeout (e.g. 60 seconds of inactivity) that falls back to the standard 2-D keyboard view.

Monitor `PowerManager.getThermalHeadroom()` ↗ ([https://developer.android.com/reference/android/os/PowerManager#getThermalHeadroom\(int\)](https://developer.android.com/reference/android/os/PowerManager#getThermalHeadroom(int))) on API 30+ and exit AR when headroom drops below `0.7`.

AR Library Comparison

CRITERION	SCENEVIEW / ARSCENEVIEW	RAW ARCORE	OPENXR (KHRONOS)
Abstraction level	High — node graph, GLB loading, built-in plane renderer	Low — raw pose & point cloud only	Medium — cross-vendor XR runtime
Rendering pipeline	Filament (PBR, IBL, bloom out of the box)	BYO renderer (OpenGL ES / Vulkan)	BYO renderer (Vulkan preferred)
Min SDK / API	API 24 + ARCore 1.38+	API 24 + ARCore 1.38+	Vendor-specific (Meta, Qualcomm)
Library size	≈ 8 MB (ARSceneView + Filament)	≈ 1 MB (ARCore client only)	≈ 3 MB (loader + runtime)
Hand tracking	Not built-in (roadmap)	Not available on phones	Supported on XR headsets
Best fit	Rapid prototyping, keyboard overlays	Custom pipelines needing full control	Cross-platform XR headset apps

Depth API Occlusion

On devices that support the [ARCore Depth API](https://developers.google.com/ar/develop/depth) ↗ (<https://developers.google.com/ar/develop/depth>), real-world objects can occlude virtual key caps, creating a more convincing illusion. Enable depth occlusion by configuring the session with `DepthMode.AUTOMATIC`.

```
fun configureARSession(session: Session) {  
    val config = session.config.apply {  
        depthMode = if  
(session.isDepthModeSupported(Config.DepthMode.AUTOMATIC))  
            Config.DepthMode.AUTOMATIC  
        else  
            Config.DepthMode.DISABLED  
    }  
}
```

KOTLIN

```
planeFindingMode = Config.PlaneFindingMode.HORIZONTAL
lightEstimationMode = Config.LightEstimationMode.ENVIRONMENTAL_HDR
updateMode = Config.UpdateMode.LATEST_CAMERA_IMAGE
}
session.configure(config)
}
```



Future — Hand Tracking Keyboards

Android XR (announced 2024) brings hand-tracking APIs for headset-class devices. A hand-tracked AR keyboard replaces raycasting with finger-tip collision against virtual key volumes. The key placement grid in `buildKeyboardNode()` already works as-is — only the input dispatch path changes from screen touch → raycast to hand joint → sphere-box collision. Watch [ARCore release notes](https://developers.google.com/ar) ↗ (<https://developers.google.com/ar>) for phone-side hand tracking availability.

EXTERNAL RESOURCES

- [ARCore Developer Guide](https://developers.google.com/ar/develop) ↗ (<https://developers.google.com/ar/develop>)
- [SceneView for Android \(GitHub\)](https://github.com/SceneView/sceneview-android) ↗ (<https://github.com/SceneView/sceneview-android>)
- [ARCore Depth API](https://developers.google.com/ar/develop/depth) ↗ (<https://developers.google.com/ar/develop/depth>)
- [PowerManager Thermal Headroom](https://developer.android.com/reference/android/os/PowerManager#getThermalHeadroom(int))) ↗ ([https://developer.android.com/reference/android/os/PowerManager#getThermalHeadroom\(int\)\)](https://developer.android.com/reference/android/os/PowerManager#getThermalHeadroom(int)))

AI-Driven Personalization

● EXPERT

On-device ML for adaptive themes, predictive color selection, typing pattern detection, and federated learning principles

The next frontier of keyboard personalization uses on-device machine learning to adapt the keyboard's appearance to the user's context, habits, and preferences — without sending data off-device. This section covers practical, production-ready approaches using Android's on-device ML stack.

Adaptive Theme Selection — TensorFlow Lite

A lightweight TFLite model can observe typing session features (time of day, app context, typing speed, error rate) and recommend a theme variant — brighter for daylight, darker for night, simpler layouts when error rate is high.

```
// Gradle
implementation("org.tensorflow:tensorflow-lite:2.14.0")
implementation("org.tensorflow:tensorflow-lite-support:0.4.4")

// Theme recommendation model input tensor: [7 features]
// [hour_sin, hour_cos, typing_speed_norm, error_rate, app_type,
//  ambient_brightness, session_duration_norm]

class ThemeAdvisor(context: Context) {

    private val interpreter: Interpreter

    init {
        val model = FileUtil.loadMappedFile(context, "theme_advisor.tflite")
        interpreter = Interpreter(model)
    }

    fun recommendTheme(features: ThemeFeatures): String {
        val input = floatArrayOf(
            sin(features.hourOfDay * Math.PI / 12).toFloat(),
            cos(features.hourOfDay * Math.PI / 12).toFloat(),
            (features.typingSpeed / 80f).coerceIn(0f, 1f),

```

GRADLE

```

        features.errorRate.coerceIn(0f, 1f),
        features.appTypeFactor,
        (features.ambientBrightness / 255f),
        (features.sessionDurationSec / 300f).coerceIn(0f, 1f)
    )
    val output = Array(1) { FloatArray(4) } // 4 theme classes
    interpreter.run(input, output)

    val themeIndex = output[0].indexOfFirst { it == output[0].max() }
    return listOf("light_standard", "dark_standard",
        "light_minimal", "dark_minimal")[themeIndex]
}
}

```

ONNX Runtime Mobile Alternative

ONNX Runtime Mobile [↗ \(https://onnxruntime.ai/docs/tutorials/mobile/\)](https://onnxruntime.ai/docs/tutorials/mobile/) offers the same on-device inference capabilities as TFLite with broader model ecosystem support (PyTorch, scikit-learn, XGBoost exports). The same `ThemeAdvisor` logic ports directly.

```
implementation("com.microsoft.onnxruntime:onnxruntime-android:1.18.0")
```

GRADLE

```

class OnnxThemeAdvisor(context: Context) {
    private val session: OrtSession

    init {
        val env = OrtEnvironment.getEnvironment()
        val opts = OrtSession.SessionOptions().apply {
            addNnapi() // Hardware acceleration via NNAPI
            setOptimizationLevel(OrtSession.SessionOptions.OptLevel.ALL_OPT)
        }
        val modelBytes = context.assets.open("theme_advisor.onnx").readBytes()
        session = env.createSession(modelBytes, opts)
    }

    fun recommendTheme(features: ThemeFeatures): String {
        val env = OrtEnvironment.getEnvironment()
        val input = floatArrayOf(/* same 7 features */)
        val tensor = OnnxTensor.createTensor(
            env, FloatBuffer.wrap(input), longArrayOf(1, 7)
        )
        val result = session.run(mapOf("input" to tensor))
    }
}

```

KOTLIN

```

        val scores = (result[0].value as Array<FloatArray>)[0]
        tensor.close()
        result.close()

        val idx = scores.indices.maxBy { scores[it] }
        return THEME_NAMES[idx]
    }

    companion object {
        private val THEME_NAMES = listOf(
            "light_standard", "dark_standard",
            "light_minimal", "dark_minimal"
        )
    }
}

```

Feature Extraction Pipeline

The 7-feature input tensor is assembled from aggregate session statistics, ambient sensors, and app context. No individual keystrokes are recorded — only derived metrics.

```

data class ThemeFeatures(
    val hourOfDay: Float,           // 0-23, encoded as sin/cos pair
    val typingSpeed: Float,         // Characters per minute
    val errorRate: Float,          // Backspaces / total keys ratio
    val appTypeFactor: Float,      // 0.0 (messaging) ... 1.0 (IDE)
    val ambientBrightness: Float,  // Lux reading from TYPE_LIGHT sensor
    val sessionDurationSec: Float  // Seconds since session start
)

object FeatureCollector {

    private var keyCount    = 0
    private var errorCount = 0
    private var sessionStart = SystemClock.elapsedRealtime()

    fun onKeyCommitted() { keyCount++ }
    fun onBackspace()    { errorCount++ }

    fun snapshot(sensorLux: Float, packageName: String): ThemeFeatures {
        val now = Calendar.getInstance()
        val elapsed = (SystemClock.elapsedRealtime() - sessionStart) / 1000f
        val cpm = if (elapsed > 0) keyCount / (elapsed / 60f) else 0f
        return ThemeFeatures(

```

KOTLIN

```

        hourOfDay      = now.get(Calendar.HOUR_OF_DAY).toFloat(),
        typingSpeed    = cpm,
        errorRate      = if (keyCount > 0) errorCount.toFloat() / keyCount
    else 0f,

    appTypeFactor      = classifyApp(packageName),
    ambientBrightness  = sensorLux,
    sessionDurationSec = elapsed
)
}

fun reset() { keyCount = 0; errorCount = 0; sessionStart =
SystemClock.elapsedRealtime() }

private fun classifyApp(pkg: String): Float = when {
    pkg.contains("messaging") || pkg.contains("whatsapp") → 0.0f
    pkg.contains("mail") || pkg.contains("gmail")         → 0.3f
    pkg.contains("browser")                               → 0.5f
    pkg.contains("editor") || pkg.contains("ide")          → 1.0f
    else                                                    → 0.5f
}
}

```

Model Quantization for Mobile

Quantising the theme advisor model reduces file size and inference latency on ARM devices. The table below compares options available in both TFLite and ONNX Runtime toolchains.

QUANTIZATION	PRECISION	SIZE REDUCTION	LATENCY IMPACT	ACCURACY LOSS	TOOLCHAIN
FP32 (baseline)	32-bit float	1×	1×	None	—
FP16	16-bit float	≈ 50 %	0.7–0.9×	Negligible	TFLite converter, ONNX quantize_static
Dynamic INT8	8-bit integer (weights only)	≈ 75 %	0.5–0.7×	Low (< 1 %)	TFLite post-training, ONNX dynamic quantization
Full INT8	8-bit integer (weights + activations)	≈ 75 %	0.4–0.6×	Moderate (1–3 %)	TFLite representative dataset, ONNX calibration

```
# Post-training dynamic quantization (TFLite)
```

PYTHON

```
import tensorflow as tf
```

```
converter = tf.lite.TFLiteConverter.from_saved_model("saved_model/")
```

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

```
converter.target_spec.supported_types = [tf.float16]
```

```
tflite_quant = converter.convert()
```

```
with open("theme_advisor_fp16.tflite", "wb") as f:
```

```
    f.write(tflite_quant)
```

On-Device Incremental Learning

Rather than shipping a static model, use **on-device training** ↗ (https://www.tensorflow.org/lite/examples/on_device_training/overview) to fine-tune weights from user corrections. The approach below stores a small correction buffer and trains in a background `WorkManager` job once per day.

```
class ThemeLearner(private val db: CorrectionDao) {
```

KOTLIN

```
    data class Correction(
        val features: FloatArray,
        val chosenThemeIndex: Int
    )
```

```
    suspend fun recordCorrection(features: ThemeFeatures, chosenTheme: String) {
        val idx = THEME_NAMES.indexOf(chosenTheme)
```

```

        if (idx < 0) return
        db.insert(CorrectionEntity(features.toArray(), idx))
    }

    suspend fun retrain(interpreter: Interpreter) {
        val corrections = db.getRecent(limit = 200)
        if (corrections.size < 20) return // Not enough data

        // Micro-batch SGD: 5 epochs, learning rate 0.001
        corrections.chunked(16).forEach { batch →
            val inputs = batch.map { it.features }.toTypedArray()
            val labels = batch.map { it.chosenThemeIndex }.toIntArray()
            // Use TFLite training API (SignatureRunner "train")
            interpreter.runSignature(
                mapOf("features" to inputs, "labels" to labels),
                mapOf(),
                "train"
            )
        }
    }
}

```



Privacy Guard

On-device training must

never persist raw keystrokes, typed text, or personally identifiable data

. The `Correction` record above stores only 7 aggregate float features and a theme index — no text content. Encrypt the local correction database with [EncryptedFile](https://developer.android.com/reference/androidx/security/crypto/EncryptedFile) [↗] (<https://developer.android.com/reference/androidx/security/crypto/EncryptedFile>) or [SQLCipher](https://www.zetetic.net/sqlcipher/sqlcipher-for-android/) [↗] (<https://www.zetetic.net/sqlcipher/sqlcipher-for-android/>) and wipe it when the user clears app data.

ML Framework Comparison

CRITERION	TENSORFLOW LITE	ONNX RUNTIME MOBILE	PYTORCH MOBILE (EXECUTORCH)
Model format	<code>.tflite</code> (FlatBuffer)	<code>.onnx</code> (Protobuf)	<code>.pte</code> (ExecuTorch) / <code>.ptl</code>
HW delegates	NNAPI, GPU Delegate, Hexagon DSP	NNAPI, CoreML, XNNPACK	XNNPACK, Vulkan, CoreML
On-device training	Yes (SignatureRunner)	Experimental	No (inference only)
Library size (AAR)	≈ 1.5 MB	≈ 2.8 MB	≈ 4.0 MB
Quantization support	FP16, INT8, INT4	FP16, INT8, INT4 (QDQ)	FP16, INT8
Ecosystem	TensorFlow / Keras / JAX	PyTorch, scikit-learn, XGBoost	PyTorch only

A/B Testing Theme Recommendations

Evaluate whether AI recommendations actually improve user satisfaction by running a controlled experiment. Hold out a control group that receives no recommendation, and compare theme-switch rates and session duration.

```
enum class ExperimentGroup { CONTROL, AI_RECOMMEND }  
  
object ThemeExperiment {  
  
    private val group: ExperimentGroup by lazy {  
        // Deterministic assignment from install ID hash  
        val hash = UUID.randomUUID().hashCode().absoluteValue  
        if (hash % 2 == 0) ExperimentGroup.CONTROL  
        else ExperimentGroup.AI_RECOMMEND  
    }  
}
```

KOTLIN

```

    }

    fun shouldShowRecommendation(): Boolean =
        group == ExperimentGroup.AI_RECOMMEND

    fun logEvent(event: String, params: Map<String, Any>) {
        // Send to analytics (group tag included automatically)
        Analytics.log("theme_experiment", params + ("group" to group.name))
    }
}

```

Privacy-Preserving Personalization Principles

- **Never log keystrokes.** Derive features (typing speed, error rate) from aggregate statistics, not individual key events.
- **All ML inference on-device.** No data leaves the device. Use TFLite or ONNX Runtime Mobile, not cloud APIs.
- **User control first.** AI recommendations are suggestions — the user's explicit theme choice always overrides the model.
- **Federated learning for future models.** If improving models from usage, use TensorFlow Federated ↗ (<https://www.tensorflow.org/federated>) — trains on-device, only aggregated gradients (never raw data) are shared.

EXTERNAL RESOURCES

- LiteRT (TensorFlow Lite) ↗ (<https://ai.google.dev/edge/litert>)
- ONNX Runtime Android ↗ (<https://onnxruntime.ai/docs/tutorials/mobile/>)
- ExecuTorch (PyTorch Mobile) ↗ (<https://pytorch.org/executorch/stable/>)
- EncryptedFile (Jetpack Security) ↗ (<https://developer.android.com/reference/androidx/security/crypto/EncryptedFile>)

Live Wallpaper Background Keys • INTERMEDIATE

Syncing the keyboard background with the device's live wallpaper colors and motion via WallpaperManager

Android's `WallpaperManager` provides APIs to extract the dominant colors of the current wallpaper and — on supported devices — to receive notifications when the wallpaper changes. Combined with Material You's `DynamicColors`, this enables a keyboard that truly **adapts its entire visual identity to the device wallpaper in real time**.

Extracting Wallpaper Colors

```
KOTLIN
@RequiresApi(27)
fun extractWallpaperPalette(context: Context): WallpaperColors? {
    val wm = WallpaperManager.getInstance(context)
    return wm.getWallpaperColors(WallpaperManager.FLAG_SYSTEM)
}

@RequiresApi(27)
fun KeyboardTheme.Companion.fromWallpaper(context: Context): KeyboardTheme {
    val colors = extractWallpaperPalette(context)
    val primary      = colors?.primaryColor?.toArgb() ?: Color.GRAY
    val secondary    = colors?.secondaryColor?.toArgb() ?: Color.LTGRAY
    val isDark       = colors?.colorHints?.and(
        WallpaperColors.HINT_SUPPORTS_DARK_THEME) != 0

    return KeyboardTheme(
        id = "wallpaper_sync",
        name = "Wallpaper Sync",
        keyBackground = colorToHex(blendColors(primary, if (isDark)
0xFF1A1A1A.toInt() else 0xFFF5F5F5.toInt(), 0.85f)),
        keyLabel      = colorToHex(if (isDark) Color.WHITE else Color.BLACK),
        keyBorder      = colorToHex(secondary),
        keyLabelPressed = colorToHex(primary),
        actionBackground = colorToHex(primary),
        actionLabel     = colorToHex(if (isDark) Color.BLACK else Color.WHITE),
        boardBackground = colorToHex(secondary))
}
```

```

    )
}

// Listen for wallpaper changes and update theme live
val wallpaperChangeReceiver = object : BroadcastReceiver() {
    override fun onReceive(ctx: Context, intent: Intent) {
        if (intent.action == Intent.ACTION_WALLPAPER_CHANGED) {
            reloadThemeFromWallpaper()
        }
    }
}
}

```

OnColorsChangedListener (API 27+)

The `BroadcastReceiver` approach above detects wallpaper *replacement* only. For smooth real-time response — including live wallpapers that shift hue continuously — register an **OnColorsChangedListener** ↗ (<https://developer.android.com/reference/android/app/WallpaperManager.OnColorsChangedListener>) that fires whenever the dominant palette changes.

```

@RequiresApi(27)
class WallpaperColorWatcher(
    private val context: Context,
    private val onColorsChanged: (WallpaperColors?) → Unit
) {
    private val wm = WallpaperManager.getInstance(context)
    private val handler = Handler(Looper.getMainLooper())

    private val listener =
        WallpaperManager.OnColorsChangedListener { colors, which →
            if (which == WallpaperManager.FLAG_SYSTEM) {
                onColorsChanged(colors)
            }
        }

    fun start() {
        wm.addOnColorsChangedListener(listener, handler)
        // Emit current colours immediately
        onColorsChanged(wm.getWallpaperColors(WallpaperManager.FLAG_SYSTEM))
    }

    fun stop() {

```

KOTLIN

```

        wm.removeOnColorsChangedListener(listener)
    }
}

```



Lock Screen vs Home Wallpaper

`FLAG_SYSTEM` targets the home screen wallpaper. `FLAG_LOCK` targets the lock screen wallpaper (may differ). For keyboard theming, `FLAG_SYSTEM` is almost always correct since the keyboard appears over apps, not the lock screen. If the user has the same wallpaper on both, `getWallpaperColors(FLAG_LOCK)` returns the same result as `FLAG_SYSTEM`.

Material You Dynamic Color Integration

On API 31+ devices, **Material You** [↗ \(https://developer.android.com/develop/ui/views/theming/dynamic-colors\)](https://developer.android.com/develop/ui/views/theming/dynamic-colors) automatically generates a tonal palette from the wallpaper. Rather than parsing `WallpaperColors` manually, use the platform's `DynamicColors` tokens for maximum visual consistency with the rest of the system.

```

@RequiresApi(31)
fun applyDynamicColors(context: Context): KeyboardTheme {
    // Resolve Material You tokens from the current wallpaper palette
    val themed = DynamicColors.wrapContextIfAvailable(context)
    val ta = themed.obtainStyledAttributes(intArrayOf(
        com.google.android.material.R.attr.colorPrimary,
        com.google.android.material.R.attr.colorSecondaryContainer,
        com.google.android.material.R.attr.colorOnSecondaryContainer,
        com.google.android.material.R.attr.colorSurface,
        com.google.android.material.R.attr.colorOnSurface,
        com.google.android.material.R.attr.colorTertiary
    ))
    val primary      = ta.getColor(0, Color.GRAY)
    val container    = ta.getColor(1, Color.LTGRAY)
    val onContainer  = ta.getColor(2, Color.BLACK)
    val surface      = ta.getColor(3, Color.WHITE)
    val onSurface    = ta.getColor(4, Color.BLACK)
    val tertiary     = ta.getColor(5, Color.GRAY)
    ta.recycle()

    return KeyboardTheme(
        id          = "material_you",
        name        = "Material You",
        keyBackground = colorToHex(container),

```

KOTLIN

```

        keyLabel      = colorToHex(onContainer),
        keyBorder      = colorToHex(tertiary),
        keyLabelPressed = colorToHex(primary),
        actionBackground = colorToHex(primary),
        actionLabel     = colorToHex(surface),
        boardBackground = colorToHex(surface)
    )
}

```

Animated Color Transitions

Snapping to new colors instantly is jarring. Animate the transition over 400–600 ms using `ValueAnimator` on each color channel so the keyboard smoothly morphs as the wallpaper changes.

```

object ColorTransitioner {
    fun animateThemeSwitch(
        view: View,
        from: KeyboardTheme,
        to: KeyboardTheme,
        durationMs: Long = 500L
    ) {
        val evaluator = ArgbEvaluator()
        ValueAnimator.ofFloat(0f, 1f).apply {
            duration = durationMs
            interpolator = FastOutSlowInInterpolator()

            addUpdateListener { anim →
                val fraction = anim.animatedFraction

                val bg = evaluator.evaluate(
                    fraction,
                    Color.parseColor(from.boardBackground),
                    Color.parseColor(to.boardBackground)
                ) as Int

                val keyBg = evaluator.evaluate(
                    fraction,
                    Color.parseColor(from.keyBackground),
                    Color.parseColor(to.keyBackground)
                ) as Int

                view.setBackgroundColor(bg)
            }
        }
    }
}

```

```

        // Apply keyBg to individual key drawables via callback
        (view as? ThemeAnimatable)?.onThemeFrame(bg, keyBg, fraction)
    }
    start()
}
}
}

interface ThemeAnimatable {
    fun onThemeFrame(boardColor: Int, keyColor: Int, fraction: Float)
}

```

Contrast Validation

Wallpaper-derived colors may produce low-contrast key labels. Check against WCAG 2.2 contrast minimum (AA = 4.5 : 1) ↗ (<https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html>) and fall back to guaranteed-contrast colors when the ratio is too low.

```

object ContrastChecker {
    fun contrastRatio(fg: Int, bg: Int): Double {
        val lum1 = relativeLuminance(fg) + 0.05
        val lum2 = relativeLuminance(bg) + 0.05
        return maxOf(lum1, lum2) / minOf(lum1, lum2)
    }

    fun ensureContrast(
        label: Int,
        background: Int,
        minRatio: Double = 4.5
    ): Int {
        if (contrastRatio(label, background) ≥ minRatio) return label
        // Fall back to black or white depending on background luminance
        return if (relativeLuminance(background) > 0.179) Color.BLACK else
Color.WHITE
    }

    private fun relativeLuminance(color: Int): Double {
        fun channel(c: Int): Double {
            val s = c / 255.0
            return if (s ≤ 0.04045) s / 12.92
            else Math.pow((s + 0.055) / 1.055, 2.4)
        }
        return 0.2126 * channel(Color.red(color)) +

```

KOTLIN

```

        0.7152 * channel(Color.green(color)) +
        0.0722 * channel(Color.blue(color))
    }
}

```



Live Wallpaper Rate Limiting

Some live wallpapers shift colors every frame. Without throttling, `OnColorsChangedListener` could trigger theme rebuilds dozens of times per second. Debounce color change callbacks to **at most once per 2 seconds**

using `Handler.postDelayed()` or a Kotlin `Flow.debounce(2000)`.

Wallpaper Source Comparison

API	MIN SDK	PROVIDES	FIRES ON	BEST FOR
<code>ACTION_WALLPAPER_CHANGED</code>	1	Intent only (no colors)	Static wallpaper replacement	Legacy fallback
<code>WallpaperColors</code>	27	Primary / secondary / tertiary + hints	Static and live wallpaper color change	Custom palette extraction
<code>DynamicColors</code> (Material 3)	31	Tonal palette (13 tones × 5 accent groups)	System palette recalculation	Full Material You alignment
<code>WallpaperManager.getDrawable()</code>	1	Full bitmap	Manual poll	Blur / custom background processing

Edge Cases & Robustness

- **No wallpaper permission:** `getWallpaperColors()` returns `null` if `READ_EXTERNAL_STORAGE` is not granted (API < 33) or if the user has disabled the wallpaper service. Always provide a default theme fallback.
- **Dark wallpaper + dark mode:** Both `HINT_SUPPORTS_DARK_THEME` and the system UI mode flag should be checked together. If the wallpaper is dark but the device is in light mode, prefer the device setting.
- **Multi-display devices:** Foldables may have separate wallpapers for inner and outer screens. Listen to `Configuration.densityDpi` changes to detect display switches and re-query colors.

- **Wallpaper not set:** Some OEMs return a solid color as the "default wallpaper". Detect this by checking `primaryColor == secondaryColor == tertiaryColor` and use a branded fallback instead.

EXTERNAL RESOURCES

- [WallpaperManager ↗](https://developer.android.com/reference/android/app/WallpaperManager) (<https://developer.android.com/reference/android/app/WallpaperManager>)
 - [Material You Dynamic Colors ↗](https://developer.android.com/develop/ui/views/theming/dynamic-colors) (<https://developer.android.com/develop/ui/views/theming/dynamic-colors>)
 - [WCAG 2.2 Contrast Minimum ↗](https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html) (<https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html>)
 - [WallpaperColors API Reference ↗](https://developer.android.com/reference/android/app/WallpaperColors) (<https://developer.android.com/reference/android/app/WallpaperColors>)
-

SECTION 28

Performance Budgets for 3D

● EXPERT

Frame timing budgets, thermal throttling management, and profiling tools for GPU-accelerated keyboards

A keyboard rendering 3D effects while the user types in a heavy app (camera, AR, game) is competing for the GPU with that app. If your keyboard causes the foreground app to drop frames, users notice immediately — even if they never consciously connect it to the keyboard. This section defines a strict performance contract for all 3D keyboard work.

Performance Budget Framework

<2ms Max GPU time per keyboard frame *Measured on Snapdragon 480*

<1ms CPU thread time per frame *All uniform uploads, state changes*

<32MB Total GPU memory for keyboard *Textures + buffers + shaders*

60 FPS Target for animated effects *Drop to 30 FPS when thermals throttle*

≤256 Max active particles *GPU compute simulation*

3 Max draw calls per frame *Use instancing to stay within*

Thermal Throttle Detection

```
// Detect thermal throttling and reduce visual quality
class ThermalAwareRenderer(context: Context) {

    private val powerManager =
context.getSystemService(PowerManager::class.java)
    private var qualityLevel = QualityLevel.HIGH
```

KOTLIN

```

init {
    if (Build.VERSION.SDK_INT ≥ 29) {
        powerManager.addThermalStatusListener { status →
            qualityLevel = when (status) {
                PowerManager.THERMAL_STATUS_NONE,
                PowerManager.THERMAL_STATUS_LIGHT → QualityLevel.HIGH
                PowerManager.THERMAL_STATUS_MODERATE → QualityLevel.MEDIUM
                else → QualityLevel.LOW // SEVERE/CRITICAL – drop to
Canvas
            }
            onQualityChanged(qualityLevel)
        }
    }
}

fun onQualityChanged(level: QualityLevel) {
    when (level) {
        QualityLevel.HIGH → { particlesEnabled = true; renderMode =
CONTINUOUS }
        QualityLevel.MEDIUM → { particlesEnabled = false; renderMode =
WHEN_DIRTY }
        QualityLevel.LOW → { switchToCanvasRenderer() }
    }
}

enum class QualityLevel { HIGH, MEDIUM, LOW }

```

Profiling Checklist

PROFILING TOOLS

- ✓ Profile with **Android GPU Inspector** — captures GPU frame traces on Qualcomm, Mali, and PowerVR
- ✓ Use **Perfetto** for CPU + GPU timeline correlation — identify when keyboard GL work overlaps with app frames
- ✓ Run **RenderDoc** for frame capture — inspect draw calls, shader invocations, texture bindings
- ✓ Test with **CPU Profiler in Android Studio** — measure GL thread CPU time alongside the main thread

TEST DEVICES (LOW-END)

- ✓ Snapdragon 480 (e.g., Nokia G60) — represents budget 5G market
- ✓ MediaTek Helio G85 (e.g., Samsung Galaxy A32) — very common in European mid-range
- ✓ Unisoc Tiger T618 (e.g., Lenovo Tab M10) — entry-level tablet GPU

BUDGET CHECKS

- ✓ GPU frame time $\leq 2\text{ms}$ on Snapdragon 480 at all temperatures
 - ✓ No texture uploads per frame (pre-upload in `onSurfaceCreated`)
 - ✓ Instanced drawing: all keys in ≤ 3 draw calls
 - ✓ Particle system: compute shader, ≤ 256 particles, $\leq 1\text{ms}$ total
-

QA Checklist — 3D & Visual

● INTERMEDIATE

Test matrix for visual correctness, performance compliance, accessibility parity, and theme system correctness

Ship visual effects with confidence. This checklist covers every verification category—from GL context lifecycle and shader compilation across GPU vendors to thermal compliance and accessibility parity—so that QA teams can certify a 3D keyboard build systematically rather than relying on ad-hoc visual inspection.

3D RENDERING CORRECTNESS

- ✓ 3D keys are visually correct in portrait and landscape on all test devices
- ✓ EGL context is preserved across keyboard hide/show cycles (no re-initialization flicker)
- ✓ Rotation / configuration change does not leave a blank GL surface
- ✓ Key press animation (sinkage) fires and completes within 60ms
- ✓ No z-fighting artifacts on bevel edges on any GPU vendor
- ✓ 3D rendering correctly composited over host app content (transparent background)

SHADER QUALITY

- ✓ All GLSL shaders compile without errors on Qualcomm Adreno, ARM Mali, and PowerVR
- ✓ medium precision does not produce visible banding artifacts on key labels
- ✓ Specular highlight position is stable during key press (no highlight flickering)

PARTICLE SYSTEM

- ✓ Particles are correctly culled when the keyboard is hidden
- ✓ No particle leak after 1,000 keystrokes (particle pool correctly reused)
- ✓ Particle effects disabled when device is in Low Power Mode

THEME ENGINE

- ✓ All bundled themes parse from JSON without schema errors
- ✓ Theme hot-swap (changing theme while keyboard is visible) applies without crash or flicker
- ✓ Material You theme correctly updates when system wallpaper changes
- ✓ User-custom background is correctly blurred and cropped to keyboard dimensions
- ✓ Font packs load correctly; invalid font path falls back to system font gracefully

ACCESSIBILITY PARITY

- ✓ TalkBack correctly announces key labels when 3D mode is active
- ✓ Key contrast meets WCAG 2.2 AA in all bundled themes ($\geq 4.5:1$ label : background)
- ✓ No visual effects (particles, glow) prevent touch target identification

PERFORMANCE

- ✓ GPU frame time $\leq 2\text{ms}$ on Snapdragon 480 at steady state
- ✓ GPU frame time $\leq 2\text{ms}$ after 10 minutes continuous typing (thermal sustained)
- ✓ Quality degrades gracefully under thermal throttling (particles $\rightarrow$ Canvas fallback)

Automated Visual Regression Testing

Manual visual inspection does not scale. Use screenshot-based regression to catch rendering regressions on every pull request. [Android Instrumented Testing](https://developer.android.com/training/testing/instrumented-tests) [↗] (<https://developer.android.com/training/testing/instrumented-tests>) combined with a pixel-diffing library catches subtle shader and compositing changes.

```
@RunWith(AndroidJUnit4::class)
@LargeTest
class KeyboardVisualRegressionTest {

    @get:Rule
    val activityRule = ActivityScenarioRule(TestInputActivity::class.java)

    @Test
    fun keyboardRendersCorrectlyInPortrait() {
```

KOTLIN

```

// Wait for GL surface initialisation
Thread.sleep(1500)

activityRule.scenario.onActivity { activity →
    val keyboardView = activity.findViewById<View>(R.id.keyboard_root)
    val screenshot = Bitmap.createBitmap(
        keyboardView.width, keyboardView.height,
        Bitmap.Config.ARGB_8888
    )
    PixelCopy.request(
        activity.window, Rect(
            keyboardView.left, keyboardView.top,
            keyboardView.right, keyboardView.bottom
        ),
        screenshot, { result →
            assertEquals(PixelCopy.SUCCESS, result)
        },
        Handler(Looper.getMainLooper())
    )

    // Compare against golden reference stored in test assets
    val golden = BitmapFactory.decodeStream(
        InstrumentationRegistry.getInstrumentation()
            .context.assets.open("golden/portrait_default.png")
    )
    val diff = pixelDiffPercentage(screenshot, golden)
    assertTrue(
        "Pixel diff $diff% exceeds 0.5% threshold",
        diff < 0.5
    )
}

private fun pixelDiffPercentage(a: Bitmap, b: Bitmap): Double {
    require(a.width == b.width && a.height == b.height)
    var diffCount = 0
    for (y in 0 until a.height) {
        for (x in 0 until a.width) {
            if (a.getPixel(x, y) != b.getPixel(x, y)) diffCount++
        }
    }
    return diffCount.toDouble() / (a.width * a.height) * 100.0
}
}

```



Golden Image Management

Store golden PNGs per GPU vendor and API level in

`src/androidTest/assets/golden/<vendor>/<api>/`. Different GPUs produce slightly different anti-aliasing and rounding, so a single golden per test is not realistic. Use `Build.HARDWARE` or `GLES20.glGetString(GLES20.GL_RENDERER)` to select the correct golden set at runtime.

Macrobenchmark: Frame Timing Verification

Use **Macrobenchmark** [↗ \(https://developer.android.com/topic/performance/benchmarking/macrobenchmark-overview\)](https://developer.android.com/topic/performance/benchmarking/macrobenchmark-overview) to measure actual frame durations under realistic typing workloads. This catches GPU stalls, excessive draw calls, and thermal degradation that unit tests cannot reveal.

```
@RunWith(AndroidJUnit4::class)
class KeyboardFrameBenchmark {

    @get:Rule
    val benchmarkRule = MacrobenchmarkRule()

    @Test
    fun typingFrameTiming() {
        benchmarkRule.measureRepeated(
            packageName = "com.example.keyboard",
            metrics = listOf(FrameTimingMetric()),
            iterations = 5,
            startupMode = StartupMode.WARM,
            setupBlock = {
                pressHome()
                startActivityAndWait()
            }
        ) {
            // Simulate 30 keystrokes at 150ms intervals (200 WPM typing)
            val letters = "the quick brown fox jumps over"
            letters.forEach { ch →
                device.pressKeyCode(ch.code)
                Thread.sleep(150)
            }
        }
    }
}
```

KOTLIN

```
    }  
  }  
}
```

GPU Vendor Test Matrix

Shader compilation, floating-point precision, and texture sampling behavior vary significantly across GPU vendors. The following matrix defines the minimum device coverage for a confident release.

GPU VENDOR	EXAMPLE SOC	DRIVER QUIRKS	TEST PRIORITY	GL ES VERSION
Qualcomm Adreno	Snapdragon 4 Gen 2 / 8 Gen 3	Occasional <code>mediump</code> precision issues in fragment shaders	PO — 45% market	3.2
ARM Mali	Dimensity 9300 / Exynos 2400	Tile-based deferred rendering; overdraw penalised heavily	PO — 38% market	3.2
Samsung Xclipse (AMD)	Exynos 2200 / 2400 (select regions)	RDNA architecture — different shader compiler from Mali	P1	3.2
PowerVR	Dimensity 700 (budget)	Aggressive FP16 quantisation; banding in gradients	P1 — budget devices	3.2
Google Tensor	Tensor G4 (Mali-G715 variant)	Custom thermal curves; earlier governor throttling	PO — Pixel flagship	3.2

ADB Diagnostic Commands

Use these commands during manual QA or in CI device farms to extract real-time performance and thermal telemetry from test devices.

```
# GPU rendering profile — frame times per window  
adb shell dumpsys gfxinfo com.example.keyboard framestats  
  
# Thermal headroom — values closer to 1.0 indicate imminent throttling  
adb shell dumpsys thermalservice | grep -A5 "Temperature"  
  
# GPU memory usage (Qualcomm Adreno)  
adb shell cat /sys/class/kgsl/kgsl-3d0/gpubusy  
  
# Surface Flinger frame stats — missed vsync count  
adb shell dumpsys SurfaceFlinger --latency com.example.keyboard
```

```
# Reset frame stats before each test run
adb shell dumpsys gfxinfo com.example.keyboard reset

# Meminfo to detect native heap leaks from GL buffers
adb shell dumpsys meminfo com.example.keyboard | grep -E "Native|Graphics"

# Force thermal throttling simulation (requires root / API 30+ emulator)
adb shell cmd thermalservice override-status 2
```



CI Emulators Have No GPU

Standard CI emulators (non-GPU) use

SwiftShader

software rendering. Frame timing numbers from emulators are meaningless for GPU performance validation. Use [Firebase Test Lab](https://firebase.google.com/docs/test-lab) or a self-hosted device farm for reliable GPU metrics. Emulators are still useful for shader compilation smoke tests and visual regression screenshots.

CI Pipeline Integration

The following GitHub Actions workflow runs shader compilation checks on every pull request and delegates GPU benchmarks to a nightly device-farm run.

```
# .github/workflows/visual-qa.yml
name: Visual QA
on:
  pull_request:
    paths: ['app/src/main/res/raw/**', 'app/src/main/assets/shaders/**']
  schedule:
    - cron: '0 3 * * *' # Nightly at 03:00 UTC

jobs:
  shader-compile-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Compile GLSL shaders
        run: |
          for f in app/src/main/assets/shaders/*.glsl; do
            glslangValidator --target-env opengl "$f" || exit 1
          done
```

YAML

```

screenshot-regression:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Set up JDK
      uses: actions/setup-java@v4
      with:
        java-version: '21'
        distribution: 'temurin'
    - name: Run screenshot tests on emulator
      uses: reactivecircus/android-emulator-runner@v2
      with:
        api-level: 34
        profile: pixel_7
        script: ./gradlew :app:connectedDebugAndroidTest -
Pandroid.testInstrumentationRunnerArguments.class=com.example.keyboard.KeyboardV
isualRegressionTest

```

```

device-farm-benchmark:
  if: github.event_name == 'schedule'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build benchmark APKs
      run: ./gradlew :benchmark:assembleRelease :app:assembleRelease
    - name: Upload to Firebase Test Lab
      uses: google-github-actions/firebase-test-lab@v1
      with:
        app-apk: app/build/outputs/apk/release/app-release.apk
        test-apk: benchmark/build/outputs/apk/release/benchmark-release.apk
        devices: |
          model=oriole,version=34
          model=a]52,version=33
          model=redfin,version=33

```

Shader Compilation Smoke Test

Catch GPU-vendor-specific compilation failures at runtime before the keyboard is shown to the user. Load-and-compile each shader on first boot and report failures to analytics.

```

object ShaderSmokeTest {

    data class ShaderResult(
        val name: String,
        val compiled: Boolean,
        val log: String
    )

    fun compileAllShaders(
        context: Context,
        shaderDir: String = "shaders"
    ): List<ShaderResult> {
        val results = mutableListOf<ShaderResult>()
        val assets = context.assets.list(shaderDir) ?: return results

        for (file in assets) {
            val source = context.assets.open("$shaderDir/$file")
                .bufferedReader().readText()

            val type = when {
                file.endsWith(".vert") → GLES20.GL_VERTEX_SHADER
                file.endsWith(".frag") → GLES20.GL_FRAGMENT_SHADER
                else → continue
            }

            val shader = GLES20.glCreateShader(type)
            GLES20.glShaderSource(shader, source)
            GLES20.glCompileShader(shader)

            val status = IntArray(1)
            GLES20.glGetShaderiv(shader, GLES20.GL_COMPILE_STATUS, status, 0)
            val log = GLES20.glGetShaderInfoLog(shader)
            GLES20.glDeleteShader(shader)

            results.add(ShaderResult(file, status[0] == GLES20.GL_TRUE, log))
        }
        return results
    }
}

```

Accessibility Contrast Audit Script

Automate WCAG 2.2 AA contrast verification for every bundled theme instead of relying on manual color-picker checks.

KOTLIN

```

@Test
fun allThemesMeetWcagContrastMinimum() {
    val themes = ThemeRegistry.allBundledThemes()
    val failures = mutableListOf<String>()

    for (theme in themes) {
        val bg = Color.parseColor(theme.keyBackground)
        val label = Color.parseColor(theme.keyLabel)
        val ratio = ContrastChecker.contrastRatio(label, bg)

        if (ratio < 4.5) {
            failures.add("${theme.id}: ratio=${"%0.2f".format(ratio)} (need
≥4.5)")
        }
    }
    assertTrue(
        "Themes failed WCAG AA contrast:\n${failures.joinToString("\n")}",
        failures.isEmpty()
    )
}

```

Thermal & Battery Test Protocol

3D keyboard effects consume significantly more GPU power than flat designs. Define a structured test protocol to verify thermal compliance on reference devices.

PHASE	DURATION	ACTION	PASS CRITERIA
1. Cold Start	0 → 30 s	Open keyboard, hold idle	GPU frame ≤ 2 ms, no jank frames
2. Rapid Typing	30 s → 5 min	Continuous typing at 200 WPM (simulated)	CPU < 15%, battery drain ≤ 3%
3. Sustained Load	5 → 15 min	Typing + particle effects + shader animations	Skin temp ≤ 40 °C, no thermal warnings
4. Throttled	15 → 20 min	Continue load under simulated thermal state 2	Graceful degradation to Canvas fallback
5. Recovery	20 → 25 min	Stop typing, keyboard idle	Return to full 3D within 60 s of cool-down



TalkBack Announcement Checklist

3D visual effects must never interfere with `contentDescription` or `AccessibilityNodeInfo`. After enabling TalkBack, verify that: (1) every key announces its character or label, (2) swipe gestures between keys produce correct traversal order, (3) popup key previews announce alternate characters, and (4) particle and glow effects do not create spurious accessibility events. Run the [Accessibility Scanner](https://developer.android.com/guide/topics/ui/accessibility/testing) on each theme before release.

EXTERNAL RESOURCES

- [Jetpack Macrobenchmark](https://developer.android.com/topic/performance/benchmarking/macrobenchmark-overview)
- [Firebase Test Lab](https://firebase.google.com/docs/test-lab)
- [glslangValidator Reference Compiler](https://github.com/KhronosGroup/glslang)
- [Android Accessibility Testing](https://developer.android.com/guide/topics/ui/accessibility/testing)

Open Source Project Index ● BEGINNER

Complete annotated index of all referenced open source projects, rendering libraries, and tools

Every open-source project, rendering library, and tool mentioned throughout this volume—indexed by category, license, and the section where it first appears—so you can evaluate dependencies before adding them to your build.

PROJECT	CATEGORY	LICENSE	PRIMARY USE IN THIS GUIDE
FlorisBoard (https://github.com/florisboard/florisboard)	Keyboard IME	Apache 2.0	Sections 03, 19 — Snygg theme engine architecture reference
AnySoftKeyboard (https://github.com/AnySoftKeyboard/AnySoftKeyboard)	Keyboard IME	Apache 2.0	Section 03 — 3D theme pack, external APK theme loading
OpenBoard (https://github.com/openboard-team/openboard)	Keyboard IME	GPL 3.0	Section 03 — LatinIME rendering pipeline, layout system
kBoard (LiteKite) (https://github.com/LiteKite/Android-IME)	Keyboard IME	Apache 2.0	Section 03 — MVVM IME architecture pattern
Android-CustomKeyboard (https://github.com/DonBrody/Android-CustomKeyboard)	Custom KB	MIT	Section 17 — Simplest custom keyboard starting point
Custom_Android_Keyboard (https://github.com/shutterscripter/Custom_Android_Keyboard)	Custom KB	MIT	Section 17 — Layout XML customization patterns
Filament (Google) (https://github.com/google/filament)	3D Renderer	Apache 2.0	Sections 09–12 — Full PBR renderer, materials, IBL
SceneView (https://github.com/SceneView/scenview-android)	3D/AR Lib	Apache 2.0	Sections 11, 25 — Filament + Compose bridge, AR keyboard
OpenGL ES (Android) (https://developer.android.com/guide/topics/graphics/opengl)	GPU API	System	Sections 04–08, Section 16 — Core 3D rendering, shaders, particles
Godot Android Library (https://docs.godotengine.org/en/stable/tutorials/platform/android/android_library.html)	Game Engine	MIT	Section 24 — Godot scene embedded in IME
TensorFlow Lite (https://www.tensorflow.org/lite)	On-device ML	Apache 2.0	Section 26 — Theme recommendation model

References & Further Reading

● BEGINNER

All official documentation, research sources, and tutorials consulted in this volume

The official documentation, research papers, and community tutorials that informed every recommendation in this volume. Bookmark the Android and GPU-specific pages—they are updated more frequently than any printed guide.

Android Official Documentation

[Creating an Input Method — Android Developers](https://developer.android.com/develop/ui/views/touch-and-input/creating-input-method) (<https://developer.android.com/develop/ui/views/touch-and-input/creating-input-method>)

developer.android.com/develop/ui/views/touch-and-input/creating-input-method

[Control and Animate the Software Keyboard](https://developer.android.com/develop/ui/views/layout/sw-keyboard) (<https://developer.android.com/develop/ui/views/layout/sw-keyboard>)

developer.android.com/develop/ui/views/layout/sw-keyboard

[Handle Keyboard Input Style](https://developer.android.com/develop/ui/views/touch-and-input/keyboard-input/style) (<https://developer.android.com/develop/ui/views/touch-and-input/keyboard-input/style>)

developer.android.com/develop/ui/views/touch-and-input/keyboard-input/style

[OpenGL ES — Android Developers Guide](https://developer.android.com/guide/topics/graphics/opengl) (<https://developer.android.com/guide/topics/graphics/opengl>)

developer.android.com/guide/topics/graphics/opengl

[RuntimeShader API Reference — AGSL](https://developer.android.com/reference/android/graphics/RuntimeShader) (<https://developer.android.com/reference/android/graphics/RuntimeShader>)

developer.android.com/reference/android/graphics/RuntimeShader

[PowerManager.addThermalStatusListener — API 29+](https://developer.android.com/reference/android/os/PowerManager#addThermalStatusListener(java.util.concurrent.Executor,android.os.PowerManager.OnThermalStatusChangedListener))) ([https://developer.android.com/reference/android/os/PowerManager#addThermalStatusListener\(java.util.concurrent.Executor,android.os.PowerManager.OnThermalStatusChangedListener\)\)](https://developer.android.com/reference/android/os/PowerManager#addThermalStatusListener(java.util.concurrent.Executor,android.os.PowerManager.OnThermalStatusChangedListener))))

developer.android.com/reference/android/os/PowerManager

Material You Dynamic Colors — Android Developers (<https://developer.android.com/develop/ui/views/theming/dynamic-colors>)

developer.android.com/develop/ui/views/theming/dynamic-colors

3D Rendering Libraries

Filament — Google PBR Rendering Engine (<https://github.com/google/filament>)

github.com/google/filament

Filament Design Documentation (<https://google.github.io/filament/Filament.html>)

google.github.io/filament/Filament.html

SceneView — Filament + ARCore + Compose (<https://github.com/SceneView/scenview-android>)

github.com/SceneView/scenview-android

Sceneform FilamentEngineWrapper Reference (<https://developers.google.com/scenform/reference/com/google/ar/scenform/rendering/FilamentEngineWrapper>)

developers.google.com/scenform/reference

Custom Keyboard Tutorials & Examples

Android Custom Keyboard Layout — AndroidDvlpr (<https://androiddvlpr.com/android-custom-keyboard-layout/>)

androiddvlpr.com/android-custom-keyboard-layout/

Custom Android Keyboard Practical Example — CodezUp (<https://codezup.com/custom-android-keyboard-practical-example/>)

codezup.com/custom-android-keyboard-practical-example/

Custom Keyboard IMF Deep Dive — CodezUp (<https://codezup.com/custom-android-keyboard-input-methods-framework/>)

codezup.com/custom-android-keyboard-input-methods-framework/

Custom Keyboard in Android Studio (Java) — CodezUp (<https://codezup.com/custom-android-keyboard-android-studio-java/>)

codezup.com/custom-android-keyboard-android-studio-java/

Custom Keyboard in Compose — Maksym Koval (Medium) (<https://medium.com/@maksymkoval1/implementation-of-a-custom-soft-keyboard-in-android-using-compose-b8522d7ed9cd>)
medium.com/@maksymkoval1 — June 2025

Open Source IME References

FlorisBoard (<https://github.com/florisboard/florisboard>)
github.com/florisboard/florisboard

AnySoftKeyboard (<https://github.com/AnySoftKeyboard/AnySoftKeyboard>)
github.com/AnySoftKeyboard/AnySoftKeyboard

OpenBoard (<https://github.com/openboard-team/openboard>)
github.com/openboard-team/openboard

kBoard — LiteKite/Android-IME (<https://github.com/LiteKite/Android-IME>)
github.com/LiteKite/Android-IME

Android-CustomKeyboard — DonBrody (<https://github.com/DonBrody/Android-CustomKeyboard>)
github.com/DonBrody/Android-CustomKeyboard

Custom_Android_Keyboard — shutterscripter (https://github.com/shutterscripter/Custom_Android_Keyboard)
github.com/shutterscripter/Custom_Android_Keyboard

AnySoftKeyboard 3D Theme — F-Droid (https://f-droid.org/en/packages/com.anysoftkeyboard.theme.three_d/)
f-droid.org/en/packages/com.anysoftkeyboard.theme.three_d/

Game Engine Integration

Godot Android Library — Official Docs (https://docs.godotengine.org/en/stable/tutorials/platform/android/android_library.html)
docs.godotengine.org/en/stable/tutorials/platform/android/android_library.html

Unity as an Android Library — Unity Docs (<https://docs.unity3d.com/Manual/UnityasaLibrary-Android.html>)

docs.unity3d.com/Manual/UnityasALibrary-Android.html

OpenGL ES & GLSL

[GLSL ES 3.20 Specification — Khronos](https://registry.khronos.org/OpenGL/specs/es/3.2/GLSL_ES_Specification_3.20.pdf) (https://registry.khronos.org/OpenGL/specs/es/3.2/GLSL_ES_Specification_3.20.pdf)

registry.khronos.org/OpenGL/specs/es/3.2/

[How to Use OpenGL ES in Android Apps — Envato Tuts+](https://code.tutsplus.com/how-to-use-opengl-es-in-android-apps--cms-28464t) (<https://code.tutsplus.com/how-to-use-opengl-es-in-android-apps--cms-28464t>)

code.tutsplus.com/how-to-use-opengl-es-in-android-apps

CITATION

How to Cite This Guide

If you reference this guide in academic work, documentation, or project READMEs, please use one of the formats below. Both are consistent with the output generated by GitHub's "Cite this repository" button and the [cffconvert](https://github.com/citation-file-format/cffconvert) (<https://github.com/citation-file-format/cffconvert>) tool from the repository's `CITATION.cff` file.

APA (7th Edition)

Made in Jurgistan. (2026). Android Keyboard Design Guide: 3D & Personalization (Version 2026.2.0) [Computer software]. <https://Made-in-Jurgistan.github.io/android-keyboard-design-guide-3d-personalization/>

BibTeX

```
@software{Made_in_Jurgistan_3D_Personalization_2026,  
  author = {Made in Jurgistan},  
  month = {1},  
  title = {{Android Keyboard Design Guide: 3D & Personalization}},  
  url = {https://Made-in-Jurgistan.github.io/android-keyboard-design-guide-3d-personalization/},  
  version = {2026.2.0},  
  year = {2026}  
}
```



GitHub "Cite this repository" button

A `CITATION.cff` file at the repository root also enables GitHub's built-in "Cite this repository" button (right sidebar on the repo page), which auto-generates APA and BibTeX citations from the same metadata. This button appears on github.com only — it does not render on the published GitHub Pages site, which is why the citation blocks above are included here.

Android Keyboard Design Guide: 3D & Personalization

3D & Personalization — 31 Sections · OpenGL ES · Filament PBR · AGSL · Particle Systems · Full Kotlin/GLSL Code

Version 2026.2.0 · Published 2026-01-15 · Updated 2026-01-15 (ISO 8601)

Made in Jurgistan — Complete Edition

Research: github.com/google/filament (<https://github.com/google/filament>) · developer.android.com (<https://developer.android.com>)

github.com/florisboard (<https://github.com/florisboard/florisboard>) ·

github.com/AnySoftKeyboard (<https://github.com/AnySoftKeyboard/AnySoftKeyboard>)

github.com/SceneView (<https://github.com/SceneView>) · docs.godotengine.org (<https://docs.godotengine.org>)

Licensed under CC BY-SA 4.0 — Free to share and adapt with attribution

Accessibility: Conforms to WCAG 2.2 AA. Includes keyboard navigation, skip links, focus-visible outlines, reduced-motion support, forced-colors support, and semantic HTML. Report issues to: github.com/Made-in-Jurgistan/android-keyboard-design-guide-3d-personalization/issues

↑ Return to top